

MEMORIA PROYECTO “Kilometro a Kilometro”

IES Cañada de la Encina

Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma (2º DAM)

Autor: Cristóbal Delgado Peñaranda

Curso: 2º DAM

Aplicación para la gestión del consumo de combustible y control de gastos vehiculares

“Controla tu gasto, conduce con cabeza”

Kilómetro a Kilómetro



Índice:

1. Descripción de la idea del proyecto.....	3
1.1 Objetivos del proyecto.....	3
1.2 Justificación y utilidad.....	4
2. Entorno.....	5
2.1 Análisis del entorno.....	5-6
2.2 Estimación del mercado.....	6-9
2.3 Análisis DAFO.....	10-13
2.4 Estudio CAME.....	14-15
3. Plan de marketing.....	16-17
3.1 Producto.....	17
3.2 Precio.....	18-19
3.3 Promoción.....	20- 21
4. Plan de negocio.....	21
4.1 Recursos humanos.....	21
4.2 Inversión y financiación.....	22-23
5. Plan jurídico y mercantil.....	23-24

6. Ayudas y financiación.....	24
7. Modelo CANVAS del proyecto.....	25-26
8. Estado del arte tecnológico y normativa.....	27
8.1 Tecnologías disponibles.....	28
8.2 Normativas aplicables.....	29
8.3 Justificación de tecnologías elegidas.....	30
9. Solución técnica del proyecto.....	30
9.1 Entorno de desarrollo y tecnología.....	31-33
9.2 Casos de uso.....	33-36
9.3 Persistencia de los datos.....	36-40
10. Diagramas y diseño del sistema.....	40
10.1 Diagrama de componentes y MVC.....	41-44
10.2 Diagrama entidad-relación.....	44-51
10.3 Diagramas de flujo.....	51-60
10.4 Diagrama de despliegue.....	60-65
10.5 Interfaces.....	65-72
11. Metodología y planificación.....	72
11.1 Estimación de costes.....	73
11.2 Diagrama de Gantt.....	74
12. Pruebas y validación.....	75-76

1.Descripcion de la idea del proyecto:

1.1 Introducción:

El presente proyecto titulado **Kilometro a Kilometro** consiste en el desarrollo de una aplicación de escritorio orientada a la gestión de consumo de combustible y control de gasto de uno o varios vehículos.

Su finalidad es ofrecer a los usuarios una herramienta sencilla que les permita registrar repostajes, calcular consumos medios y visualizar la evolución de su gasto en gasolina o diésel de forma clara y organizada.

La aplicación está pensada para conductores particulares y pequeños negocios que deseen tener un control detallado de su consumo de el combustible.

1.2 Objetivos del proyecto:

El objetivo principal del proyecto es crear una aplicación práctica y visual que permita al usuario gestionar su gasto de manera eficiente.

Los objetivos específicos son:

- Registrar repostajes con datos como fecha litros precio y Kilometraje.
- Calcular automáticamente el consumo medio del vehículo.
- Mostrar un historial de gastos y estadísticas visuales.
- Permitir la gestión de vehículos desde una misma cuenta.

Incluir un sistema básico de registro y acceso seguro de usuario.

1.3 Justificación:

La idea de *Kilómetro a Kilómetro* surge ante la falta de herramientas simples y accesibles que permitan a los usuarios gestionar su consumo de combustible con control directo sobre sus datos. Aunque existen aplicaciones como [Fuelio](#) o [Drivvo](#), la mayoría están centradas en móviles y dependen de la nube.

El proyecto comienza con una versión de escritorio por ser un entorno cómodo para la gestión y el análisis, pero está concebido para evolucionar hacia **versiones móviles o web**, manteniendo una arquitectura flexible y escalable. La aplicación combina trabajo local con funciones online opcionales, ofreciendo un equilibrio entre privacidad, comodidad y expansión futura.

2. Entorno:

El proyecto *Kilómetro a Kilómetro* se enmarca en el ámbito del **transporte y la gestión del consumo de combustible**, un sector que en los últimos años ha experimentado un notable interés debido al **aumento del precio de los carburantes**, la **conciencia medioambiental** y la **digitalización de los procesos de control de gastos**.

En este contexto, cada vez más usuarios buscan herramientas tecnológicas que les permitan **monitorizar sus consumos y gastos de forma precisa y accesible**.

2.1 Análisis de entorno:

Macroentorno:

Justificación Económica:

El contexto económico actual, marcado por la **volatilidad en el precio de los combustibles**, ha generado una preocupación creciente entre los conductores. Según los informes del Ministerio para la Transición Ecológica, el coste del combustible ha experimentado variaciones constantes en los últimos años, afectando directamente al presupuesto mensual de miles de usuarios.

En este escenario, disponer de herramientas que permitan **controlar el gasto por kilómetro, comparar precios en estaciones cercanas y optimizar los desplazamientos** se ha convertido en una necesidad real. La aplicación *Kilómetro a Kilómetro* responde a esta problemática proporcionando un sistema accesible que calcula consumos, identifica gasolineras más económicas y ayuda al usuario a tomar decisiones basadas en datos actualizados.

Este tipo de soluciones también favorece la **transparencia del mercado**, promoviendo una competencia más justa entre estaciones de servicio y beneficiando al consumidor final.

Justificación Tecnológica

El desarrollo de *Kilómetro a Kilómetro* es viable gracias a la **madurez actual de herramientas y lenguajes de programación**, como Python, y el uso de librerías orientadas a interfaces gráficas, visualización de mapas y tratamiento de datos. Estas tecnologías permiten crear aplicaciones de escritorio:

- Ligeras
- Multiplataforma
- De bajo coste de mantenimiento
- Con posibilidad de funcionar sin conexión (modo offline)

Además, el acceso a **APIs abiertas**, como la ofrecida por el Geoportal de Gasolineras de España, facilita la integración de datos oficiales en tiempo real, proporcionando información fiable y verificable.

La combinación de estas herramientas con sistemas de almacenamiento local o bases de datos como SQLite garantiza la **persistencia, seguridad y eficiencia** del manejo de datos del usuario.

Justificación Social y Medioambiental

En los últimos años se ha producido un aumento significativo de la **conciencia social sobre la sostenibilidad y el impacto ambiental del transporte**, uno de los sectores con mayor emisión de CO₂.

Aplicaciones como *Kilómetro a Kilómetro* apoyan estos objetivos al permitir:

Reducir desplazamientos innecesarios gracias a la localización de estaciones más eficientes.

Promover un **conducción más responsable y planificada**, basada en el seguimiento del consumo.

Concienciar al usuario sobre la importancia de mantener hábitos de movilidad sostenibles.

De forma indirecta, la herramienta contribuye a la **reducción de emisiones** y fomenta el uso más racional del vehículo privado, alineándose con las directrices europeas de eficiencia energética y transición ecológica.

Microentorno:

Los clientes potenciales de *Kilómetro a Kilómetro* son principalmente **conductores particulares, autónomos y pequeñas empresas** que buscan controlar de manera sencilla y precisa los gastos asociados al uso del vehículo. Este perfil de usuario valora herramientas que permitan registrar consumos, calcular costes por desplazamiento, analizar hábitos de gasto y localizar estaciones de servicio cercanas.

En fases iniciales, la aplicación se desarrolla como **solución de escritorio**, ya que muchos profesionales y usuarios que gestionan sus gastos prefieren hacerlo desde el ordenador, donde trabajan con mayor comodidad y control. Sin embargo, este enfoque no excluye futuras versiones para dispositivos móviles. El diseño modular del proyecto permite una **evolución natural hacia una app móvil**, garantizando que las funcionalidades esenciales puedan adaptarse posteriormente a Android o iOS sin rediseños drásticos.

Además, *Kilómetro a Kilómetro* está orientada a usuarios que desean mantener el control de sus datos, pudiendo funcionar sin necesidad de servicios externos. No obstante, cuando existe conexión a internet, la aplicación se beneficia de funciones avanzadas como la consulta del **Geoportal de Gasolineras del Ministerio para la Transición Ecológica**, permitiendo mostrar precios oficiales, ubicaciones actualizadas y cálculos de gasto basados en información real. Esta dualidad ofrece un sistema **flexible**, útil tanto en entornos conectados como offline.

En cuanto a la competencia directa, el mercado está dominado por aplicaciones móviles como **Fuelio**, **Drivvo** o **Simply Auto**. Aunque ofrecen múltiples funcionalidades, presentan limitaciones para ciertos perfiles de usuario: dependencia constante de internet, sincronización obligatoria con la nube, modelos freemium o ausencia de herramientas equivalentes en formato de escritorio. *Kilómetro a Kilómetro* se diferencia al proporcionar una solución **más controlada, privada y adaptable**, centrada inicialmente en PC pero con un camino claro hacia la movilidad.

También existe una competencia indirecta en forma de hojas de cálculo, documentos manuales o software genérico de gestión, que aunque usados por algunos conductores, no aportan automatización, datos oficiales, mapas

interactivos ni facilidad de uso. Esto refuerza la necesidad de una herramienta especializada y moderna.

En conjunto, el microentorno evidencia que existe una **oportunidad real de mercado** para una aplicación que combine sencillez, control local de datos, acceso opcional a información oficial y una arquitectura que permita crecer hacia futuras versiones móviles, respondiendo así a las necesidades tanto de usuarios domésticos como profesionales.

2.2 Estimación de mercado

Una vez definida la orientación de **Kilómetro a Kilómetro** hacia el mercado de conductores particulares y pequeños autónomos que gestionan uno o varios vehículos, es posible cuantificar su público potencial mediante las métricas clave: **TAM, SAM y SOM**.

Estas métricas permiten determinar el alcance teórico, el mercado realmente disponible y el público objetivo inicial para el lanzamiento del proyecto.

TAM (Total Adressable Market) (Mercado total disponible):

El **TAM** representa el número total de conductores o vehículos que podrían utilizar una aplicación de gestión de combustible si no existieran limitaciones técnicas ni de acceso.

Según la Dirección General de Tráfico (DGT), en España existen más de 36 millones de vehículos registrados (turismos, motocicletas, furgonetas, camiones, etc.) a fecha de 2025.

TAM estimado: 36.000.000 de vehículos en España.

Este número refleja el total potencial del mercado automovilístico, incluyendo tanto a conductores particulares como profesionales del transporte y pequeñas flotas empresariales.

SAM (Serviceable Available Market) (Mercado disponible accesible):

El **SAM** delimita el grupo de usuarios al que la aplicación puede acceder realmente, considerando su enfoque y características.

Kilómetro a Kilómetro está dirigida especialmente a **conductores particulares, autónomos y pequeñas empresas** que deseen llevar un control del consumo de combustible.

De acuerdo con datos del **Instituto Nacional de Estadística (INE)** y la **DGT**, aproximadamente un **35 % de los conductores particulares** en España utilizan herramientas digitales (apps, hojas de cálculo, software de gestión) para controlar sus gastos o consumos.

Aplicando este porcentaje al TAM:

35 % de 36.000.000 $\approx$ 12.600.000 usuarios potenciales accesibles (SAM).

Estos usuarios representan el público que dispone de ordenador y conexión básica, y que muestra interés por el ahorro, la planificación o el mantenimiento de su vehículo.

3. SOM (Serviceable Obtainable Market) (Mercado objetivo alcanzable):

El **SOM** representa el porcentaje real del mercado al que **Kilómetro a Kilómetro** podría llegar en su primera fase.

Se estima razonable alcanzar un **0,2 % del mercado accesible (SAM)** durante el primer año, lo que equivale a aproximadamente **25.000 usuarios potenciales**.

Esta cifra permitiría validar la utilidad del proyecto y sentar la base para futuras versiones móviles o con conexión en la nube.

2.3 ANALISIS DAFO:



Debilidades (D):

- Al ser una aplicación de escritorio, su portabilidad es limitada frente a las aplicaciones móviles que dominan el mercado.
- Difusión y visibilidad reducida, al no estar disponible en plataformas de distribución como Play Store o Microsoft Store.
- Dependencia del entorno de ejecución de Python, lo que puede suponer una barrera técnica para usuarios no familiarizados con la instalación de software.
- Falta de recursos económicos y humanos para desarrollar funciones avanzadas o estrategias de marketing más amplias.

Amenazas (A):

- Ya existen aplicaciones muy conocidas como *Fuelio* o *Drivvo* que hacen algo parecido, y eso puede dificultar que nuevos usuarios se interesen por mi aplicación.
- Hoy en día la mayoría de personas utilizan su teléfono para este tipo de gestiones, por lo que una app de escritorio puede tener menos uso al principio.
- En poco tiempo pueden aparecer nuevas herramientas o sistemas más modernos que dejen obsoleta mi versión si no se actualiza.
- Al no estar publicada en tiendas como Google Play o Microsoft Store, es más difícil que la gente conozca la aplicación.

Fortalezas (F):

- Aplicación de escritorio sencilla, intuitiva y enfocada en la usabilidad, lo que facilita su adopción por parte de cualquier tipo de usuario.
- Funcionamiento local, sin depender de conexión a internet ni de servicios externos, lo que garantiza privacidad y control total de los datos.
- Desarrollo basado en tecnologías abiertas (Python y SQLite), lo que permite un mantenimiento económico y la posibilidad de futuras ampliaciones.

- Proyecto modular y escalable, con una estructura organizada que facilita la incorporación de nuevas funcionalidades como estadísticas o alertas de consumo.

Oportunidades (O):

- Aumento del interés por la eficiencia energética y el control de gastos personales, que impulsa la demanda de herramientas de seguimiento del consumo.
- Tendencia creciente hacia el uso de soluciones locales y seguras, frente a la dependencia de plataformas en la nube.
- Posibilidad de evolucionar hacia una versión multiplataforma o móvil que amplíe el alcance del proyecto y su potencial de mercado.
- Potencial de colaboración con talleres o empresas de transporte interesadas en registrar y optimizar sus gastos de combustible.

2.4 Estudio CAME:



Corregir debilidades (C):

- Optimizar el proceso de instalación, incorporando un asistente o guía sencilla para facilitar el uso a cualquier usuario.
- Diseñar una interfaz moderna e intuitiva, adaptada a diferentes tamaños de pantalla y resoluciones.
- Implementar documentación básica y un pequeño manual de ayuda dentro de la aplicación.
- Mejorar la experiencia de inicio ofreciendo mensajes claros ante errores o contraseñas incorrectas.

- Planificar una futura versión ejecutable para Windows que no requiera instalación de Python.

Afrontar las amenazas (A):

- Diferenciar el proyecto frente a las aplicaciones móviles, destacando su enfoque **local, sin conexión y más privado**.
- Implementar un plan de mejora continua con actualizaciones periódicas.
- Diseñar una posible versión ligera para dispositivos móviles o tablets en futuras fases.
- Fomentar la confianza del usuario a través de pruebas, demostraciones y campañas informativas.
- Reforzar la seguridad de los datos para competir frente a aplicaciones conectadas a la nube.

Mantener Fortalezas (M):

- Mantener la **sencillez y claridad** en el uso de la aplicación como eje principal del diseño.
- Continuar apostando por un **modelo de almacenamiento local** que garantice privacidad y control de datos.
- Seguir utilizando **tecnologías accesibles y multiplataforma** como Python y SQLite, que facilitan la evolución del proyecto.

- Conservar la estructura modular para poder añadir nuevas funciones sin comprometer el rendimiento.

Explotar las oportunidades (E):

- Desarrollar futuras versiones que integren funciones avanzadas (estadísticas gráficas, alertas, exportación a Excel, etc.).
- Adaptar el proyecto para que pueda ser utilizado en **pequeñas flotas o empresas de transporte locales**.
- Promocionar la aplicación en redes sociales, foros y comunidades de motor o ahorro energético.
- Explorar la posibilidad de incorporar un sistema de **registro opcional en la nube** que permita sincronizar datos entre dispositivos.

Buscar colaboración con entidades de formación o talleres que promuevan el uso de herramientas digitales para conductores.

3. Plan de Marketing

3.1 Producto:

Kilómetro a Kilómetro ofrece una aplicación de escritorio enfocada en la **gestión integral del consumo de combustible y control de gastos vehiculares**, orientada

tanto a conductores particulares como a pequeños negocios con uno o varios vehículos.

La plataforma se estructura en **dos grandes líneas de servicio**:

Por un lado, un sistema local de **registro y análisis de repostajes**, que permite al usuario guardar información sobre litros, precio y kilometraje, calcular automáticamente el consumo medio, generar estadísticas y visualizar la evolución de sus gastos de forma clara y ordenada.

Por otro lado, un módulo de **gestión de vehículos y visualización de datos**, que ofrece resúmenes personalizados, comparativas de consumo entre vehículos y gráficos de eficiencia energética.

Además, el sistema incluye un **panel informativo y visual**, con estadísticas actualizadas del gasto total, consumo medio y rendimiento por vehículo. En futuras versiones se prevé incorporar funciones adicionales como alertas de mantenimiento, recordatorios de ITV o exportación de informes a formatos externos.

Kilómetro a Kilómetro se presenta con una **interfaz moderna, limpia e intuitiva**, diseñada con colores sobrios y profesionales asociados al ámbito del transporte y la tecnología (tonos grises, naranjas y negros), favoreciendo la legibilidad y la facilidad de uso.

Su estructura modular permite que cualquier usuario, incluso sin conocimientos técnicos, pueda gestionar sus vehículos y consumos de forma rápida, privada y segura desde su propio ordenador.

¿Qué necesidades cubre *Kilometro a Kilometro*?

El cliente principal de *Kilómetro a Kilómetro* es el **conductor particular** que desea llevar un control detallado de su consumo de combustible y gastos asociados al uso de su vehículo.

También está orientada a **autónomos y pequeños negocios** que gestionan una o

varias furgonetas, coches de reparto o vehículos de empresa, y que buscan una herramienta sencilla para registrar repostajes

Ventajas frente a la competencia

Kilómetro a Kilómetro se diferencia por ser una **aplicación de escritorio completamente local**, orientada a usuarios que prefieren gestionar sus datos de manera privada y sin depender de servicios externos.

Además, ofrece una **mayor personalización en la visualización de estadísticas y comparativas** entre vehículos, permitiendo al usuario comprender fácilmente su consumo y sus gastos.

Esta combinación de **autonomía, privacidad y simplicidad** representa su principal ventaja competitiva dentro del sector.

3.2 Precio:

Kilómetro a Kilómetro funcionará bajo un **modelo gratuito**. El acceso a todas las funciones básicas registro de repostajes, gestión de vehículos, cálculo de consumos y visualización de estadísticas será completamente libre.

No obstante, se contempla la posibilidad de incorporar en el futuro un **modelo freemium**, en el que la versión básica continúe siendo gratuita, y una versión avanzada incluya funciones adicionales como:

- exportación de datos a formatos Excel o PDF,
- alertas automáticas de mantenimiento o ITV,
- y sincronización opcional con almacenamiento en la nube.

El precio estimado de esta versión premium sería **de entre 2,99 € y 4,99 € en pago único**, fijado en función de los costes de mantenimiento y comparado con aplicaciones de referencia como Fuelio o *Drivvo*.

Gracias a su naturaleza local y sin servidores externos, los **costes técnicos actuales son mínimos**, lo que permite mantener la gratuidad del servicio sin inversión en infraestructura ni alojamiento en la nube. Este enfoque facilita la adopción del producto y refuerza su carácter educativo y accesible.

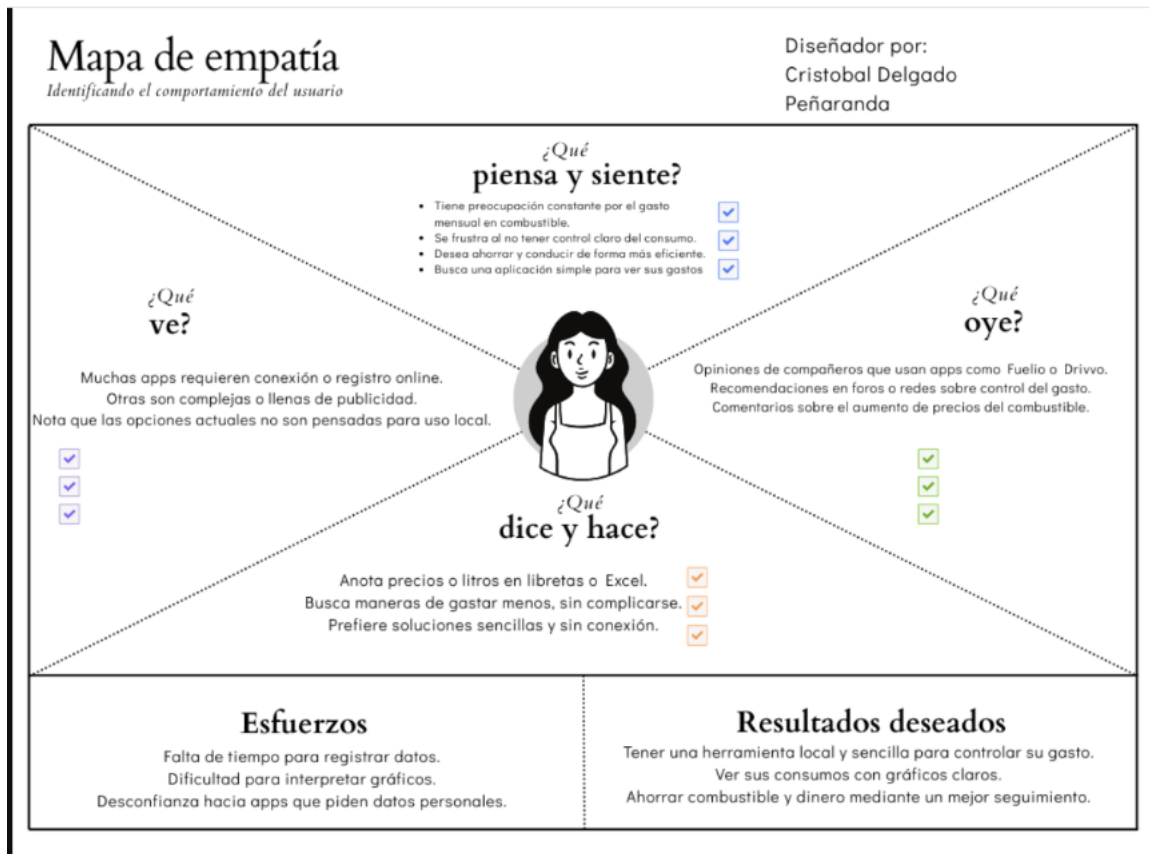
3.3 Promoción:

Nuestro público objetivo son los **conductores particulares, autónomos y pequeños negocios** que gestionan uno o varios vehículos y desean llevar un control claro de su gasto en combustible sin depender de aplicaciones en la nube.

Para entender mejor cómo piensan estos usuarios se ha elaborado un pequeño **mapa de empatía**, analizando sus motivaciones, frustraciones y expectativas en torno al control del gasto y la eficiencia en el uso del vehículo.



Mapa Empatía Usuario Kilometro a Kilometro



Plan publicitario inicial:

El plan publicitario de *Kilómetro a Kilómetro* estará centrado en dar a conocer la aplicación, atraer a los primeros usuarios y comprobar si realmente interesa al público al que va dirigida.

El objetivo principal es generar visibilidad, conseguir descargas y recoger opiniones que ayuden a mejorar futuras versiones.

Este plan tendrá una **duración de tres meses**, durante los cuales se lanzará una primera versión funcional (PMV o Producto Mínimo Viable) para probar su funcionamiento real y evaluar el impacto de la promoción.

Las principales acciones que se llevarán a cabo son las siguientes:

Meta Ads (Facebook e Instagram):

Se crearán anuncios dirigidos a personas entre 20 y 55 años, especialmente en regiones como Castilla-La Mancha, Andalucía y Comunidad Valenciana.

Los anuncios mostrarán vídeos cortos o imágenes de la aplicación, destacando su facilidad de uso, su diseño limpio y el hecho de que no necesita conexión a internet.

Google Ads (búsqueda y display):

Se harán campañas con palabras clave como “app para controlar el gasto de gasolina”, “registrar repostajes”, o “cómo ahorrar combustible”.

Estos anuncios llevarán al usuario directamente a la página de descarga o a la web del proyecto.

Cada mes se harán dos campañas por plataforma, con diferentes mensajes e imágenes adaptadas al tipo de usuario (conductores, autónomos o estudiantes).

Presupuesto estimado:

150 € al mes para Meta Ads + 150 € al mes para Google Ads = 300 €/mes.

Total, en 3 meses: 900 €

El objetivo de esta primera campaña no es ganar dinero, sino comprobar el interés de la gente, dar visibilidad al proyecto y recoger feedback real para seguir mejorando *Kilómetro a Kilómetro*.

4. Plan de Negocio:

El plan de negocio de *Kilómetro a Kilómetro* tiene como objetivo analizar la viabilidad del proyecto desde el punto de vista organizativo y económico, evaluando los recursos necesarios para su desarrollo, implantación y mantenimiento, así como su capacidad de adaptación a distintos escenarios de uso.

La aplicación se plantea como una solución con una estructura sólida y escalable, diseñada para ofrecer una funcionalidad clara desde su lanzamiento, al mismo tiempo que permite su ampliación progresiva en función de las necesidades de los usuarios y de la evolución del mercado.

4.1 Recursos Humanos:

El desarrollo de **Kilómetro a Kilómetro** se ha llevado a cabo inicialmente por una única persona, que asume de forma integral las tareas necesarias para la creación del producto. Esta organización responde a un modelo de proyecto reducido, habitual en fases tempranas de desarrollo de software.

Las funciones asumidas incluyen:

- **Análisis y diseño del sistema**, definiendo los requisitos funcionales y no funcionales de la aplicación.
- **Desarrollo del backend**, encargado de la lógica de negocio, persistencia de datos y cálculos de consumo.
- **Diseño e implementación de la interfaz gráfica**, utilizando PySide6 y Qt Designer.
- **Integración de servicios externos**, como la consulta de precios y localización de gasolineras mediante APIs oficiales.
- **Pruebas y corrección de errores**, garantizando el correcto funcionamiento del sistema.
- **Documentación técnica y funcional**, necesaria para el mantenimiento y comprensión del proyecto.

Este modelo permite un control total sobre el desarrollo y facilita la toma de decisiones técnicas. En una fase de crecimiento, el proyecto podría ampliarse

incorporando perfiles especializados como desarrolladores adicionales, diseñadores de interfaz o personal de soporte técnico.

4.2 Inversión y financiación:

La inversión inicial necesaria para el desarrollo de **Kilómetro a Kilómetro** es reducida, ya que se apoya en herramientas y tecnologías de uso gratuito o de bajo coste.

Los principales elementos de inversión son:

- **Equipo informático:** ordenador personal utilizado para el desarrollo (coste ya asumido).
- **Software de desarrollo:** Visual Studio Code, Python, PySide6 y SQLite, todos ellos gratuitos.
- **Acceso a internet:** necesario para la consulta de documentación, descarga de librerías y uso puntual de APIs externas.
- **Tiempo de desarrollo:** principal recurso invertido, destinado al diseño, programación, pruebas y documentación del proyecto.

No se requiere inversión en servidores propios ni infraestructuras en la nube, ya que la aplicación funciona principalmente de forma local. Esto reduce significativamente los costes de mantenimiento y facilita la sostenibilidad del proyecto.

En caso de una futura comercialización o ampliación, podrían considerarse nuevas fuentes de financiación como:

- reinversión de beneficios,
- modelos de pago único o versión premium,

- o ayudas públicas relacionadas con proyectos tecnológicos y de digitalización.

Este planteamiento económico permite que **Kilómetro a Kilómetro** sea un proyecto viable, escalable y adaptable a diferentes escenarios de crecimiento.

Esta estructura permite que el proyecto sea sostenible a medio plazo, con un riesgo económico reducido y una relación equilibrada entre inversión y valor aportado al usuario.

5 Plan jurídico y legal:

El proyecto *Kilómetro a Kilómetro* se plantea como una aplicación de gestión y apoyo al control del consumo de combustible, por lo que su desarrollo debe ajustarse a la normativa vigente en materia mercantil y legal, especialmente en lo relativo a la protección de datos y al uso del software.

Desde el punto de vista mercantil, el proyecto podría desarrollarse bajo la figura de **empresario individual (autónomo)** en una fase inicial, ya que se trata de una solución digital con una estructura sencilla, bajos costes de inicio y sin necesidad de una organización empresarial compleja. Esta forma jurídica permite una mayor flexibilidad en la gestión y una reducción de los trámites administrativos iniciales. En fases posteriores, y en caso de crecimiento del proyecto, podría valorarse la constitución de una sociedad mercantil.

En cuanto a la normativa legal, la aplicación debe cumplir con el **Reglamento General de Protección de Datos (RGPD)** y la **Ley Orgánica de Protección de Datos y Garantía de los Derechos Digitales (LOPDGDD)**. Aunque la aplicación prioriza el almacenamiento local de la información del usuario, cualquier tratamiento de datos personales deberá realizarse de forma segura, garantizando la confidencialidad, integridad y disponibilidad de los datos.

Asimismo, en caso de utilizar servicios externos como APIs públicas para la obtención de precios del combustible o información de gasolineras, se deberán respetar las condiciones de uso y licencias de dichas fuentes, evitando un uso indebido de la información y citando correctamente su procedencia.

Por último, el software desarrollado deberá cumplir con la normativa de **propiedad intelectual**, asegurando que el código, librerías y recursos utilizados respeten las licencias correspondientes, especialmente en el uso de herramientas de software libre o de código abierto.

6. Ayudas y financiación:

El proyecto *Kilómetro a Kilómetro* presenta unas necesidades económicas reducidas en su fase inicial, debido al uso de tecnologías gratuitas y a la ausencia de infraestructuras complejas. No obstante, en un escenario de crecimiento o comercialización, podrían considerarse distintas líneas de ayudas y financiación que permitirían mejorar el producto y ampliar su alcance.

En el ámbito público, existen diversas **ayudas y subvenciones destinadas a proyectos tecnológicos, de digitalización y emprendimiento**, tanto a nivel autonómico como estatal. Entre ellas destacan los programas de apoyo a la transformación digital, el fomento del desarrollo de software y la innovación tecnológica impulsados por organismos como:

El **Ministerio de Industria, Comercio y Turismo**, a través de programas de apoyo a la digitalización y modernización de soluciones tecnológicas.

Las **comunidades autónomas**, que suelen ofrecer subvenciones para proyectos de desarrollo de software, innovación y emprendimiento juvenil.

Programas europeos como **Next Generation EU**, orientados a impulsar la digitalización, la eficiencia energética y el uso responsable de los recursos.

Estas ayudas podrían destinarse a mejorar la interfaz de usuario, ampliar funcionalidades, optimizar el rendimiento de la aplicación o adaptar el sistema a nuevos entornos como dispositivos móviles.

Desde el punto de vista de la financiación privada, el proyecto podría sostenerse inicialmente mediante **autofinanciación**, reinvertiendo los posibles ingresos obtenidos a través de una futura versión premium o servicios adicionales. Dado que los costes de mantenimiento son bajos, este modelo resulta viable sin necesidad de grandes inversiones externas.

Otra opción sería la colaboración con **entidades relacionadas con el sector del transporte**, como talleres, pequeñas flotas o asociaciones de conductores, que podrían mostrar interés en una herramienta de control de consumo adaptada a sus necesidades. Estas colaboraciones permitirían financiar mejoras específicas del sistema a cambio de versiones personalizadas o licencias de uso.

En conclusión, *Kilómetro a Kilómetro* no depende de ayudas externas para su funcionamiento inicial, pero dispone de múltiples vías de financiación potenciales que permitirían su evolución y consolidación como una solución tecnológica sólida, escalable y sostenible en el tiempo.

7 Modelos Canvas de Kilometro a Kilometro

Modelo de Negocio Canvas Kilometro a Kilometro			
Socios Clave	Actividades clave	Recursos clave	Relacion con los clientes
- Proveedores de datos públicos (Geoportal de Gasolineras del Ministerio para la Transición Ecológica). - Plataformas de distribución de software (en fases posteriores). - Comunidades y foros de conductores y ahorro energético. - Posibles colaboradores tecnológicos (formación, talleres, autoescuelas).	- Desarrollo y mantenimiento de la aplicación. - Gestión y tratamiento de datos de consumo y repostajes. - Integración de información de gasolineras y precios oficiales. - Mejora continua de la experiencia de usuario. - Soporte y resolución de incidencias.	- Aplicación de escritorio funcional y modular. - Base de datos local (SQLite) para la gestión de usuarios, vehículos y repostajes. - Conocimiento técnico en desarrollo multiplataforma. - Tiempo de desarrollo y mantenimiento. - Herramientas de desarrollo (Python, PySide6, GitHub).	- Uso autónomo e intuitivo de la aplicación. - Mensajes claros de ayuda y validación dentro de la interfaz. - Documentación básica y tutoriales. - Atención a sugerencias y mejoras mediante feedback de usuario
Estructura de costes - Coste de desarrollo (tiempo invertido). - Equipo informático. - Acceso a internet. - Costes de promoción (en fases posteriores). - Mantenimiento y actualización del software.	Segmentos de clientes - Conductores particulares. - Autónomos que utilizan el vehículo como herramienta de trabajo. - Pequeñas empresas con uno o varios vehículos. - Usuarios interesados en el ahorro y la eficiencia energética.		Canales - Distribución directa mediante descarga. - Repositorio del proyecto (GitHub). - Redes sociales y foros relacionados con motor y ahorro. - Difusión en entornos educativos y profesionales.
	Fuentes de ingresos - Modelo gratuito en la versión básica. - Posible versión premium con funciones avanzadas: - exportación de datos, - estadísticas avanzadas, - sincronización entre dispositivos. - Pago único o modelo freemium en fases futuras.		

8. Estado del arte tecnológico y normativa

El desarrollo de aplicaciones destinadas a la gestión del consumo de combustible y al control de gastos vehiculares se apoya actualmente en un amplio conjunto de tecnologías maduras, accesibles y bien documentadas. Estas herramientas permiten crear soluciones estables, eficientes y adaptables a distintos entornos de uso.

En el ámbito tecnológico, existen numerosas aplicaciones móviles y plataformas digitales orientadas al registro de repostajes, cálculo de consumos y análisis de gastos. Sin embargo, muchas de estas soluciones presentan una elevada dependencia de servicios en la nube, modelos de suscripción o una complejidad innecesaria para usuarios que buscan un control sencillo y directo de su información.

En este contexto, Kilómetro a Kilómetro se posiciona como una solución técnica basada en tecnologías consolidadas, priorizando la claridad, la eficiencia y el control de los datos, sin renunciar a la posibilidad de incorporar servicios online cuando resulten útiles para el usuario.

8.1 Tecnologías disponibles

En la actualidad existen diversas tecnologías que permiten desarrollar aplicaciones de gestión de consumo y análisis de datos, tanto en entorno de escritorio como móvil o web.

En cuanto a lenguajes de programación, Python se ha consolidado como una de las opciones más utilizadas en proyectos de escritorio y análisis de datos, gracias a su sintaxis clara, su amplia comunidad y la gran cantidad de librerías disponibles. Lenguajes como Java, C# o Kotlin también son habituales en este tipo de aplicaciones, especialmente en entornos empresariales o móviles.

Para el desarrollo de interfaces gráficas, frameworks como Qt, JavaFX o WPF permiten crear aplicaciones visuales, modernas y multiplataforma. En el caso concreto de Python, bibliotecas como PySide6 o PyQt facilitan el uso del framework Qt, ofreciendo componentes avanzados y una integración sencilla con la lógica del programa.

Respecto a la persistencia de datos, existen múltiples sistemas gestores de bases de datos. Para aplicaciones locales y de tamaño medio, SQLite es una solución ampliamente utilizada por su ligereza, rapidez y facilidad de integración. Otros sistemas como MySQL o PostgreSQL se emplean en escenarios con servidores dedicados o aplicaciones distribuidas.

En relación con la obtención de información externa, el uso de APIs públicas se ha convertido en una práctica habitual. En el ámbito del combustible, el Geoportal de Gasolineras de España ofrece datos oficiales sobre precios y localización de estaciones de servicio, lo que permite integrar información fiable y actualizada en aplicaciones de terceros.

8.2 Normativas aplicables

El desarrollo y uso de la aplicación Kilómetro a Kilómetro debe ajustarse a la normativa vigente, especialmente en lo relativo a la protección de datos, el uso del software y la propiedad intelectual.

En primer lugar, resulta de aplicación el **Reglamento General de Protección de Datos (RGPD)** y la **Ley Orgánica de Protección de Datos y Garantía de los Derechos Digitales (LOPDGDD)**. Aunque la aplicación prioriza el almacenamiento local de la información, cualquier dato personal gestionado por el sistema debe tratarse de forma segura, garantizando la confidencialidad, integridad y disponibilidad de la información del usuario.

Asimismo, en el uso de APIs públicas, como las relacionadas con precios de carburantes, es obligatorio respetar las condiciones de uso establecidas por los organismos que proporcionan dichos datos, evitando un uso indebido de la información y citando correctamente su procedencia.

Desde el punto de vista del software, el proyecto utiliza herramientas y librerías de código abierto, por lo que es necesario cumplir con las licencias correspondientes. Python, PySide6 y SQLite cuentan con licencias permisivas que permiten su uso en proyectos académicos y comerciales, siempre que se respeten sus términos.

8.3 Justificación de las tecnologías elegidas

Las tecnologías seleccionadas para el desarrollo de Kilómetro a Kilómetro responden a criterios de eficiencia, accesibilidad y escalabilidad.

Python ha sido elegido como lenguaje principal por su claridad, facilidad de mantenimiento y amplia disponibilidad de librerías orientadas al tratamiento de datos, interfaces gráficas y persistencia. Esto permite un desarrollo ágil y una curva de aprendizaje adecuada al contexto del proyecto.

PySide6 (Qt for Python) se ha seleccionado para la construcción de la interfaz gráfica por ofrecer un entorno profesional, estable y multiplataforma, con componentes visuales avanzados y compatibilidad con el patrón de diseño MVC. Además, el uso de Qt Designer facilita la creación y modificación de interfaces de forma visual.

SQLite se utiliza como sistema gestor de base de datos por su ligereza y su integración directa con Python, permitiendo almacenar los datos de forma local sin necesidad de servidores externos. Esta elección refuerza la simplicidad del sistema y reduce los costes de mantenimiento.

Por último, el uso de APIs oficiales para la obtención de datos de gasolineras aporta un valor añadido a la aplicación, al permitir trabajar con información real, actualizada y verificable, mejorando la utilidad del sistema para el usuario final.

En conjunto, la combinación de estas tecnologías proporciona una base sólida para el desarrollo de Kilómetro a Kilómetro, garantizando estabilidad, facilidad de evolución y adecuación a las necesidades actuales y futuras del proyecto.

9 Solucion tecnica de el proyecto:

9.1 Entorno de desarrollo y tecnologia:

La aplicación *Kilómetro a Kilómetro* se apoya en un conjunto de tecnologías y herramientas orientadas al desarrollo de software de escritorio moderno, funcional y fácil de mantener.

El objetivo técnico es proporcionar una plataforma clara y estable que permita gestionar repostajes, consumos y gastos vehiculares de forma intuitiva.

Entornos de desarrollo y tecnología:

El entorno utilizado para el desarrollo ha sido **Visual Studio Code**, junto con extensiones orientadas a Python que facilitan:

- depuración del código,
- gestión de entornos virtuales,
- organización modular del proyecto.

Este entorno destaca por su flexibilidad, compatibilidad multiplataforma y amplia comunidad de soporte.

Lenguaje de programación:

La aplicación está desarrollada en **Python 3.9.13**, un lenguaje moderno y ampliamente utilizado en proyectos tanto académicos como profesionales. Python ofrece una sintaxis clara, una curva de aprendizaje rápida y un conjunto muy amplio de librerías que permiten integrar interfaces gráficas, bases de datos y cálculo estadístico de forma eficiente.

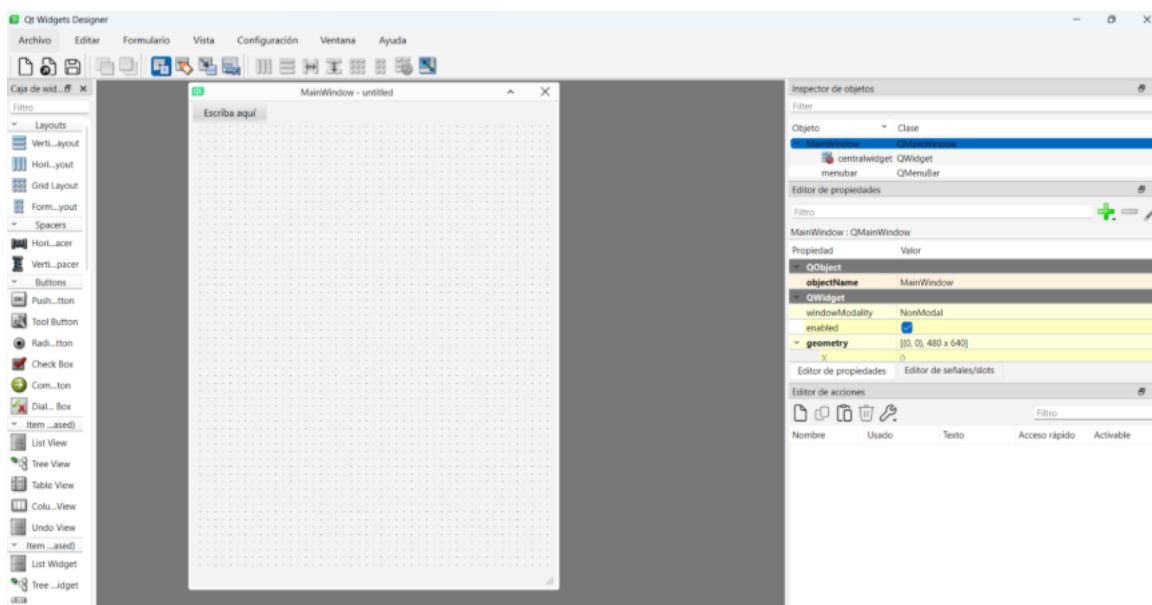
Framework de interfaz: PySide6

Para el diseño y construcción de la interfaz gráfica se ha seleccionado **PySide6 (Qt for Python)**, uno de los frameworks más completos para interfaces de usuario.

Sus principales ventajas son:

- Uso de **Qt Designer** para crear pantallas de forma visual.
- Componentes avanzados como tablas, tarjetas, botones, menús y gráficas.
- Estructura adaptada para trabajar con el patrón MVC.
- Posibilidad de ampliar fácilmente la interfaz con nuevos módulos.
- PySide6 permite construir una aplicación estable y visualmente limpia, adecuada para la gestión de datos como repostajes o consumo medio.

Aquí vemos como se empezaría a desarrollar una interfaz con el diseñador QtDesigner:



Nos ofrece una infinidad de widgets que también cabe la posibilidad de su reutilización para poder modular de manera clara nuestra aplicación.

Base de datos: SQLite

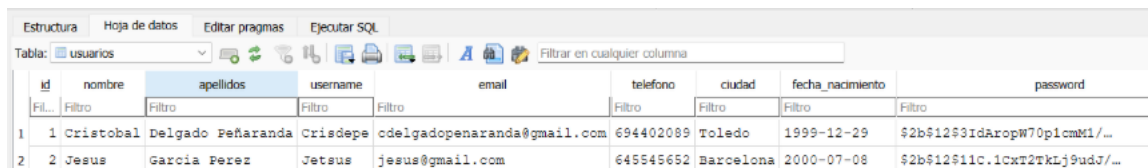
El almacenamiento de la información se realiza mediante **SQLite**, un motor de base de datos integrado directamente en Python.

Ventajas en el contexto del proyecto:

- Alto rendimiento para consultas de lectura/escritura.
- No requiere configuraciones complejas.
- Permite organizar de forma estructurada las entidades del sistema: usuarios, vehículos, repostajes y estadísticas.

Facilita migraciones futuras a otros motores de datos si el proyecto evoluciona.

Aqui vemos como se almacenarian nuestros usuarios de forma segura, con sus datos correspondientes y sus contraseñas hasheadas para proteger la identidad de el usuario y que solo el pueda tener un acceso seguro y unico.



	id	nombre	apellidos	username	email	telefono	ciudad	fecha_nacimiento	password
1	1	Cristobal	Delgado Peñaranda	Crisdepe	cdelgadopenaranda@gmail.com	694402089	Toledo	1999-12-29	\$2b\$12\$3idAropW70p1cmM1/...
2	2	Jesus	Garcia Perez	Jetsus	jesus@gmail.com	645545652	Barcelona	2000-07-08	\$2b\$12\$11c.1CxtT2TkLj9ud3/...

Aqui tambien se ve que los vehiculos que tiene cada uno de los usuarios diferenciando cual es de cada uno por el id de el usuario.

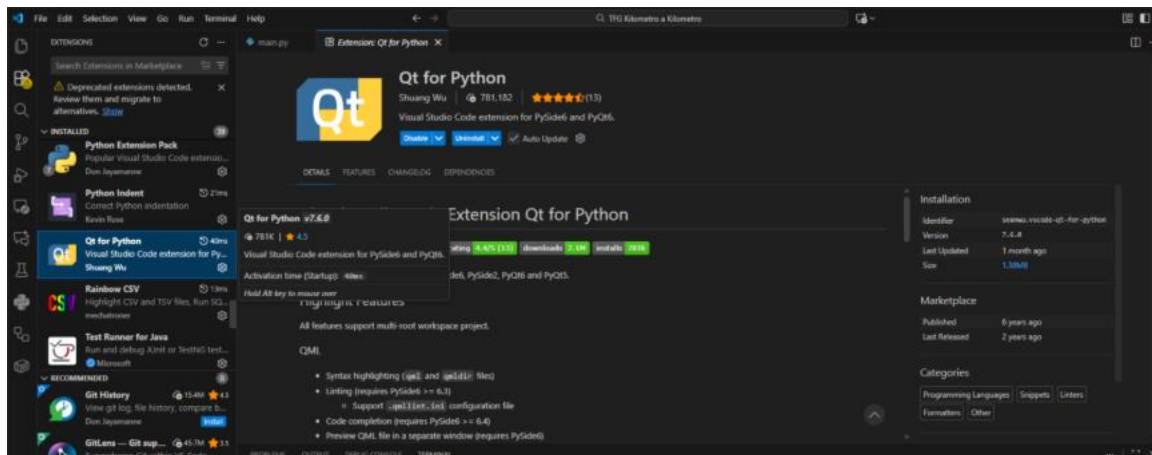


	id	user_id	tipo	marca	modelo	matricula	anio	combustible	consumo
1	1	1	Furgonetas	Peugeot	Partner	6143-JRP	2018	Gasolina	45.0
2	6	1	Motos	Triumph	Street Triple	4113-LLL	2000	Diésel	34.0

Aqui vemos algunas de las extensiones ofrecidas por Visual Studio Code para el desarrollo de la app.

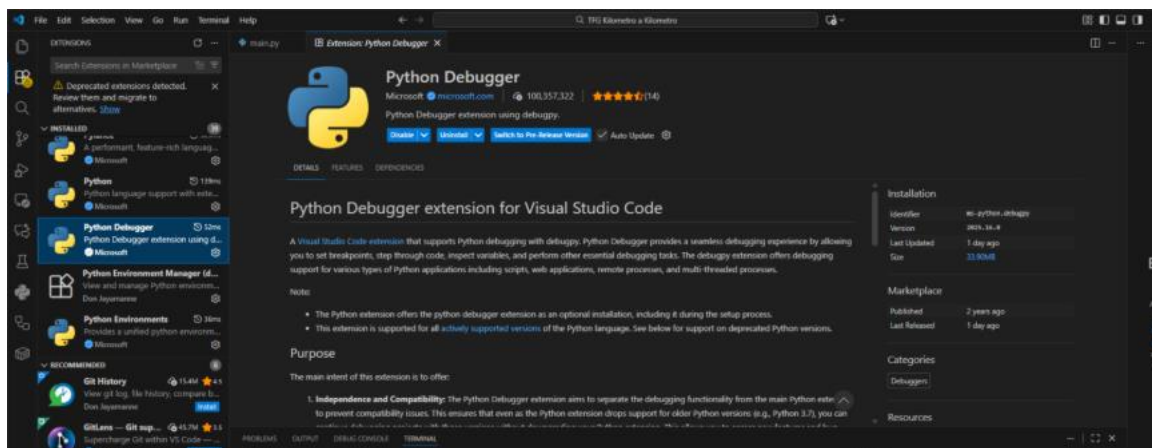
Qt for Python:

Nos permite compilar directamente nuestros archivos.ui directamente a un archivo.py totalmente listo para su uso:



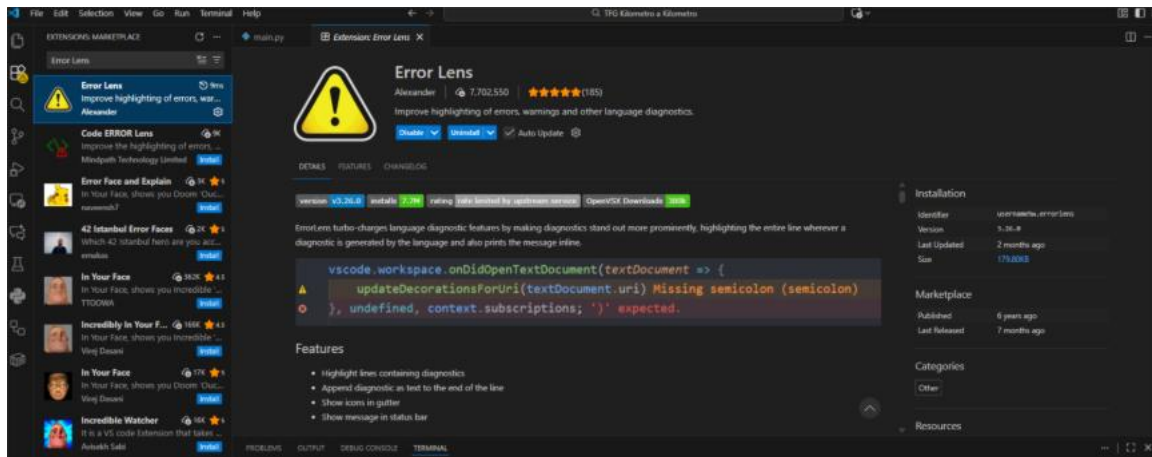
Python Debugger:

Nos permite en nuestro código realizar depuraciones completas línea a línea para poder ver si algo nos está fallando.



Error Lens:

Esta extensión lo que nos muestra es que parte de nuestro código está fallando bien sea por mala indentación o por errores a la hora de programar.



Arquitectura del software:

La arquitectura utilizada es **MVC (Modelo – Vista – Controlador)**, separando la aplicación en tres capas:

Modelo:

Contiene la lógica de datos, consultas a la base de datos, clases de usuario, vehículo y repostaje, así como los cálculos derivados (consumo medio, gasto total, estadísticas, etc.).

Vista

Conjunto de pantallas creadas con Qt Designer, donde se presenta la información mediante formularios, tarjetas, gráficos y tablas.

Controlador

Gestiona la interacción del usuario, conectando las acciones de la vista con las funciones del modelo.

Ejemplos: registrar un repostaje, actualizar estadísticas, cargar listas de vehículos...

Esta arquitectura permite mantener un código organizado y facilita la incorporación de nuevas funcionalidades en versiones posteriores lo que hace que la aplicación sea totalmente modular y escalable.

Tecnologías y librerías complementarias

- **Matplotlib / QtCharts** → para generar gráficos de consumo y evolución del gasto.
- **Pathlib / OS** → gestión de rutas internas y archivos.
- **Dataclasses** → representación clara y estructurada de objetos como vehículo o repostaje.
- **Pillow (opcional)** → gestión de imágenes e iconos.

Para el proyecto es necesario un orden y una clara estructura de carpetas aquí ves como serian esas carpetas y la logica que contienen ambas.

La carpeta **data** es aquella que realiza nuestra conexion con la base de datos en este caso recibe el nombre de usuarios.db.

```
import sqlite3

DB_NAME = "usuarios.db"

class Database:
    def __init__(self):
        self.create_tables()

    def get_connection(self):
        return sqlite3.connect(DB_NAME)

    def create_tables(self):
        con = self.get_connection()
        cur = con.cursor()

        cur.execute("""
            CREATE TABLE IF NOT EXISTS usuarios (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                nombre TEXT NOT NULL,
                apellidos TEXT NOT NULL,
                username TEXT NOT NULL UNIQUE,
                email TEXT NOT NULL UNIQUE,
                telefono TEXT NOT NULL,
                ciudad TEXT NOT NULL,
                fecha_nacimiento TEXT NOT NULL,
                password TEXT NOT NULL
            )
        """)
```

Otra carpeta que debemos distinguir es la carpeta **repositories**, donde se implementan las distintas operaciones **CRUD** del proyecto: **Create (INSERT)**,

Read (SELECT), Update y Delete. En esta capa se centraliza el acceso a los datos, permitiendo mantener un código más organizado y separando la lógica de negocio de la gestión de la base de datos.

Aquí vemos un ejemplo de como seria la inserccion de los usuarios:

```
import sqlite3

class UsuarioRepository:
    def __init__(self, db):
        self.db = db

    def insertar(self, nombre, apellidos, username, email, telefono, ciudad, fecha, password_hash):
        con = self.db.get_connection()
        cur = con.cursor()
        try:
            cur.execute("""
                INSERT INTO usuarios
                (nombre, apellidos, username, email, telefono, ciudad, fecha_nacimiento, password)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?)
            """, (nombre, apellidos, username, email, telefono, ciudad, fecha, password_hash))

            con.commit()
            return True

        except Exception as e:
            print("Error al insertar usuario:", e)
            return False

        finally:
            con.close()
```

Ahora distinguimos una de las capas más importantes de nuestra aplicación: la **capa de servicios**. Esta capa permite gestionar la lógica de negocio, actuando como intermediaria entre los *repositories* y el controlador. Gracias a ella, las operaciones se mantienen organizadas, reutilizables y desacopladas del acceso a datos.

Aquí vemos claro como actua esta capa y cual es su finalidad.

```

1 from models.Usuario import Usuario
2 import bcrypt
3 import re
4
5 class UsuarioService:
6     def __init__(self, repo):
7         self.repo = repo
8
9     # =====
10    # VALIDACIONES
11    # =====
12    def validar_email(self, email):
13        patrones = [
14            r"^[a-zA-Z0-9._%+-]+@gmail\.com$",
15            r"^[a-zA-Z0-9._%+-]+@hotmail\.com$",
16            r"^[a-zA-Z0-9._%+-]+@outlook\.com$"
17        ]
18        return any(re.match(p, email) for p in patrones)
19
20    def validar_telefono(self, tel):
21        return tel.isdigit() and 9 <= len(tel) <= 12
22
23    # =====
24    # REGISTRO
25    # =====
26    def registrar(self, nombre, apellidos, username, email, telefono, ciudad, fecha, password):
27        if not self.validar_email(email):
28            return "EMAIL_INVALIDO"
29
30        if not self.validar_telefono(telefono):
31            return "TELEFONO_INVALIDO"
32
33        hashed = bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt())
34
35        ok = self.repo.insertar(nombre, apellidos, username, email, telefono, ciudad, fecha, hashed)
36        return "OK" if ok else "ERROR"
37

```

Aqui vemos un ejemplo de como seria la capa de el controlador ya que es la gestiona la interacción del usuario, conectando las acciones de la vista con las funciones del modelo, aqui lo que hacemos es que cada ventana tenga su propio controlador con su propia logica interna quedando luego todas las ventanas a disposicion de el App Controller que actuara como cerebro de todas nuestras ventanas.

Aqui vemos como actua nuestro App Controller.

```

# =====
#   CONTROLADORES DE VENTANAS
# =====
def show_login(self):
    self.login = LoginController(self, self.user_service)
    self.login.show()

def show_registro(self):
    self.registro = RegistroController(self, self.user_service, self.cities_service)
    self.registro.show()

def open_menu(self, usuario):
    self.menu = MenuController(self, usuario)
    self.menu.show()

def open_vehiculos(self, usuario):
    """Abre la gestión de vehículos."""
    self.vehiculos = VehiculosController(
        self,
        self.vehiculo_service,
        self.vehiculo_repo,
        usuario
    )
    self.vehiculos.show()

def open_ajustes(self, usuario):
    self.ajustes = AjustesController(self, usuario)
    self.ajustes.show()

def open_perfil(self, usuario):
    self.perfil = PerfilUsuarioController(self, usuario, self.user_service, self.vehiculo_repo)
    self.perfil.show()

```

Y la ultima de las capas a mencionar es la carpeta de Vista la cual se encaraga de generar los archivos.py a traves de los archivos.ui generados por Qt-Designer:

```

s > vehiculos.ui
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
  <class>VentanaVehiculos</class>
  <widget class="QWidget" name="VentanaVehiculos">
    <property name="geometry">
      <rect>
        <x>0</x>
        <y>0</y>
        <width>700</width>
        <height>450</height>
      </rect>
    </property>
    <property name="windowTitle">
      <string>Gestión de Vehículos</string>
    </property>
    <layout class="QVBoxLayout" name="verticalLayout">
      <item>
        <widget class="QLabel" name="labelTitulo">
          <property name="text">
            <string>Mis Vehículos</string>
          </property>
          <property name="alignment">
            <set>Qt::AlignCenter</set>
          </property>
          <property name="styleSheet">
            <string>font-size:18px; font-weight:bold;</string>
          </property>
        </widget>
      </item>

      <item>
        <widget class="QTableWidget" name="tableVehiculos"/>
      </item>
    </layout>
  </widget>
</ui>

```

```

views > vehiculos_ui.py > ...
22 class VentanaVehiculos(object):
23     def setupUi(self, VentanaVehiculos):
24         if not VentanaVehiculos.setObjectName():
25             VentanaVehiculos.setObjectName(u"VentanaVehiculos")
26         VentanaVehiculos.resize(700, 450)
27         self.verticalLayout = QVBoxLayout(VentanaVehiculos)
28         self.verticalLayout.setObjectName(u"verticalLayout")
29         self.labelTitulo = QLabel(VentanaVehiculos)
30         self.labelTitulo.setObjectName(u"labelTitulo")
31         self.labelTitulo.setAlignment(Qt.AlignCenter)
32
33         self.verticalLayout.addWidget(self.labelTitulo)
34
35         self.tableVehiculos = QTableWidget(VentanaVehiculos)
36         self.tableVehiculos.setObjectName(u"tableVehiculos")
37
38         self.verticalLayout.addWidget(self.tableVehiculos)
39
40         self.horizontalLayoutButtons = QHBoxLayout()
41         self.horizontalLayoutButtons.setObjectName(u"horizontalLayoutButtons")
42         self.botonAnadir = QPushButton(VentanaVehiculos)
43         self.botonAnadir.setObjectName(u"botonAnadir")
44

```

Control de versiones:

Para el control del desarrollo se ha utilizado **Git**, junto a un repositorio alojado en **GitHub**, lo que permite:

- mantener un historial completo del desarrollo,
- organizar ramas para nuevas funciones,
- revertir errores,
- documentar cambios,
- facilitar la portabilidad del proyecto.
- Disponibles en estos Repositorio (**App de Python y App de Kotlin**):
- https://github.com/cristobaldp/Kilometro_Python
- https://github.com/cristobaldp/Kilometro_A_Kilometro_Api_Android

Tecnologías previstas para la futura versión multiplataforma

Aunque la versión actual del proyecto es de escritorio, la evolución natural del sistema contempla una adaptación a entorno móvil.

Para esa futura ampliación se valorará el uso de:

- **Kotlin** → lenguaje moderno recomendado por Google para desarrollo Android.
- **SQLite móvil** → compatible con la base ya utilizada.
- **Jetpack Compose** → para el diseño nativo de pantallas y componentes.

Esto permitirá mantener la lógica y estructura actual, adaptándola posteriormente a un entorno multiplataforma sin necesidad de rehacer la aplicación desde cero.

9.2Análisis:

Requisitos funcionales:

- Registro y Login de los Usuarios.
- Gestion de vehiculos (El usuario podra añadir, borrar, editar y consultar su informacion)

- El usuario podra introducir datos en cada repostaje (Litros repostados, Precio por litro, coste total , Kilometraje)
- Consulta de el historial de repostajes (El usuario podrá visualizar una tabla con sus repostajes ordenados por fecha, pudiendo revisar litros, coste, kilometraje y datos relacionados.
- Calculo automatico de el consumo
- Gestion de el perfil (El usuario podra modificar sus datos como por ejemplo las contraseñas , email)
- Estadisticas y graficos.

Requisitos no funcionales:

- Aplicacion fluida y estable.
- Persistencia de datos en la nube (sin necesidad de usar un servidor propio)
- Escalabilidad de el backend para futuras mejoras

Diseño e implementacion de la persistencia en los datos:

En Kilómetro a Kilómetro, la persistencia de los datos se implementa mediante SQLite, un motor de base de datos ligero, rápido y totalmente integrado con Python.

SQLite permite almacenar todos los registros de vehículos, repostajes y usuarios de forma estructurada, manteniendo la integridad y coherencia del sistema.

Para gestionar la base de datos se utiliza el modulo estandar de **sqlite3**:

Aqui vemos como se define el nombre de la base, se crea la clase y se cree la conexion con el sistema gestor de la base de datos.

```
import sqlite3

DB_NAME = "usuarios.db"

class Database:
    def __init__(self):
        self.create_tables()

    def get_connection(self):
        return sqlite3.connect(DB_NAME)
```

Tabla Usuario:

Contendra la informacion de cada usuario registrado, aqui vemos como seria la entidad de nuestros usuarios, definiendo su clave primaria y sus atributos.

```
cur.execute("""
CREATE TABLE IF NOT EXISTS usuarios (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    apellidos TEXT NOT NULL,
    username TEXT NOT NULL UNIQUE,
    email TEXT NOT NULL UNIQUE,
    telefono TEXT NOT NULL,
    ciudad TEXT NOT NULL,
    fecha_nacimiento TEXT NOT NULL,
    password TEXT NOT NULL
)
""")
```

Tabla de Vehiculo:

Almacenara los datos de cada vehiculo asociado a un usuario, con su clave primaria y su clave foranea haciendo referencia a los usuarios.

```
# Tabla de vehículos
cur.execute("""
CREATE TABLE IF NOT EXISTS vehiculos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    tipo TEXT NOT NULL,
    marca TEXT NOT NULL,
    modelo TEXT NOT NULL,
    matricula TEXT NOT NULL UNIQUE,
    anio INTEGER NOT NULL,
    combustible TEXT NOT NULL,
    consumo REAL NOT NULL,
    FOREIGN KEY (user_id) REFERENCES usuarios(id)
)
""")
```

10 Diagramas:

10.1 Diagrama de casos de uso:

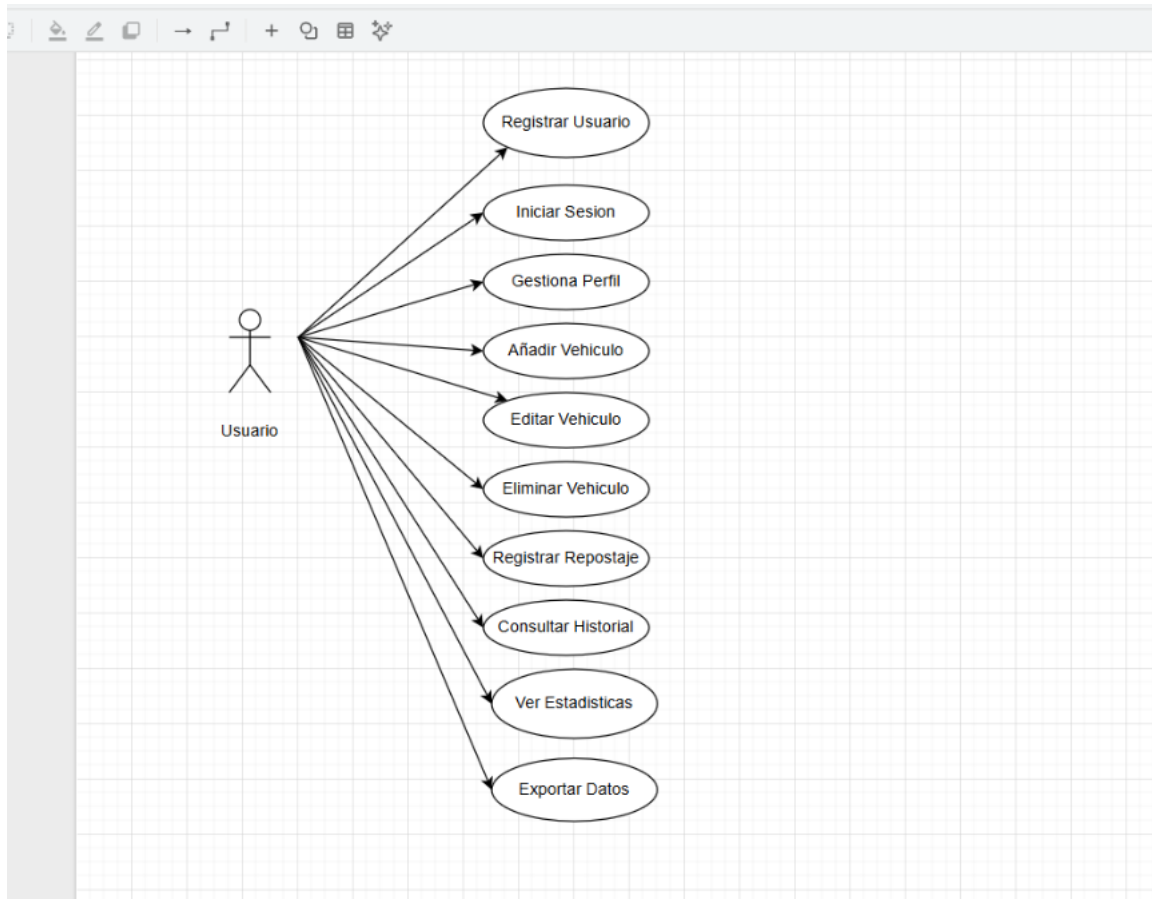


Diagrama de Componentes y estructura MVC

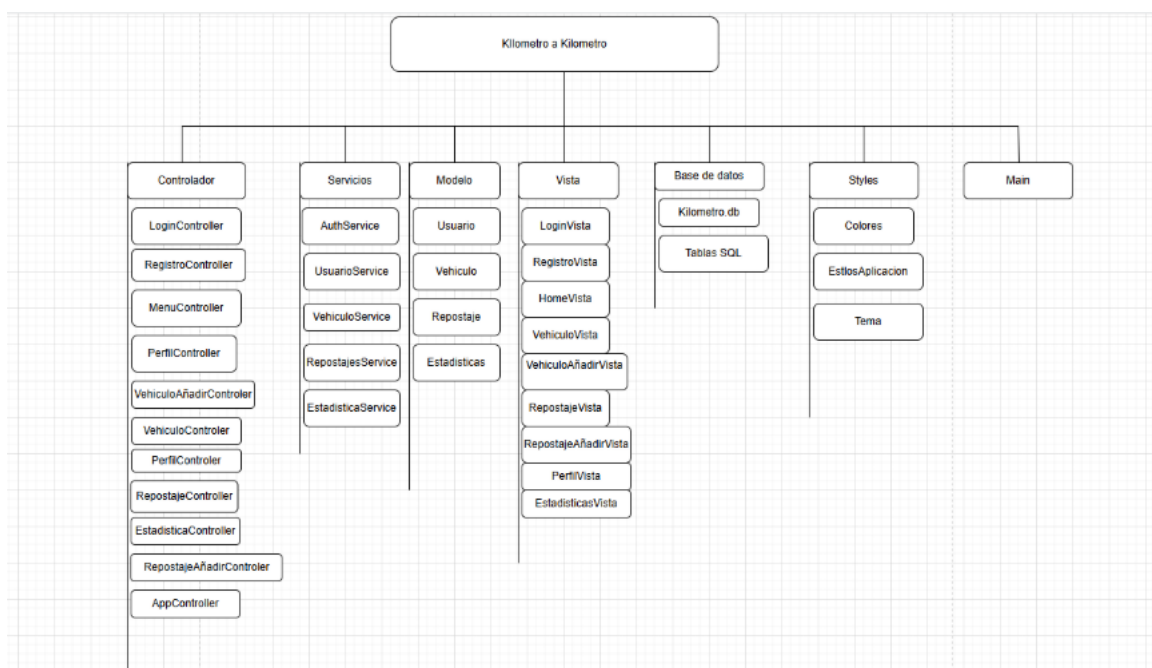
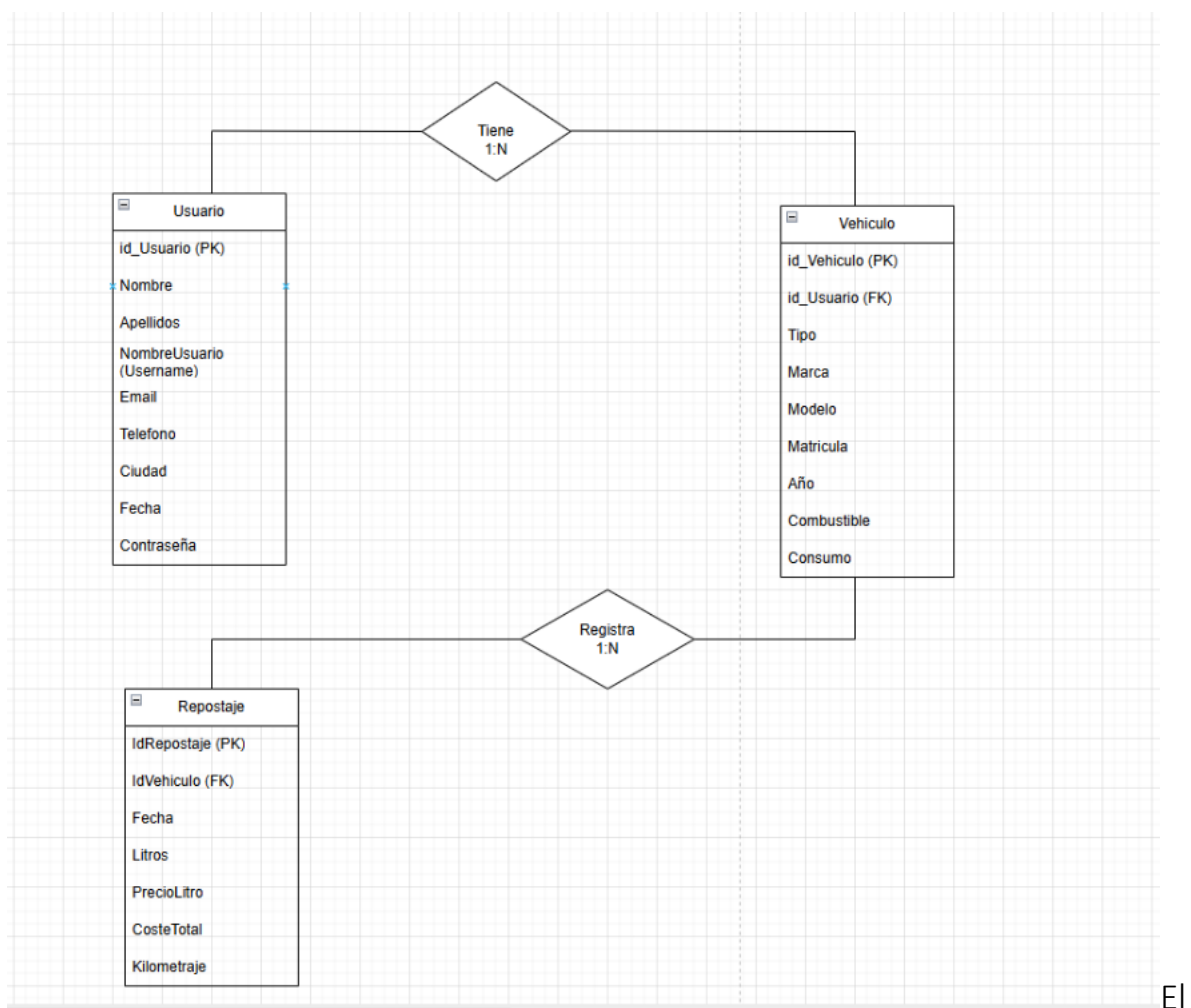


Diagrama Entidad – Relacion



El

diagrama de entidad–relación muestra cómo interactúan y se vinculan las distintas entidades que forman parte del sistema *Kilómetro a Kilómetro*. Este

diagrama representa el modelo de persistencia que utiliza la aplicación para almacenar y organizar la información, garantizando coherencia e integridad en los datos gestionados por los usuarios.

Entidad Usuario:

Un usuario mantiene su propio perfil, compuesto por atributos como nombre, apellidos, email, teléfono o ciudad. Además, la contraseña se almacena de forma cifrada para reforzar la seguridad del acceso.

Cada usuario gestiona de forma independiente sus vehículos, lo que establece una **relación 1:N entre Usuario y Vehículo**.

Es decir, un usuario puede tener varios vehículos registrados, y cada vehículo siempre pertenece a un único usuario.

Esta estructura permite que cada usuario visualice únicamente los datos asociados a su cuenta.

Relacion Usuario – Vehiculo (1:N)

La relacion “Tiene” (1: N) indica que un usuario puede añadir multiples vehiculos y cada vehiculo esta asociado a un usuario mediante el campo id_Usuario.

Esta relación es fundamental para organizar el consumo y los datos específicos de cada vehículo sin mezclar información entre distintos usuarios.

Entidad Vehiculo:

La entidad **Vehículo** almacena información clave sobre cada vehículo registrado por un usuario: marca, modelo, matrícula, combustible, año o consumo aproximado.

Esta información sirve como base para gestionar el historial de repostajes y generar estadísticas personalizadas para cada vehículo.

Relacion Vehiculo – Repostaje (1:N)

La relacion “Registra” determina que un vehiculo puede registrar uno o varios repostajes , cada repostaje está vinculado a un vehiculo mediante el IdVehiculo.

Esta relación permite identificar con precisión el comportamiento del vehículo, calcular consumos medios, observar tendencias y generar comparativas entre distintos vehículos de un mismo usuario.

Entidad Repostaje:

La entidad **Repostaje** contiene la información registrada por el usuario en cada carga de combustible. Incluye datos como:

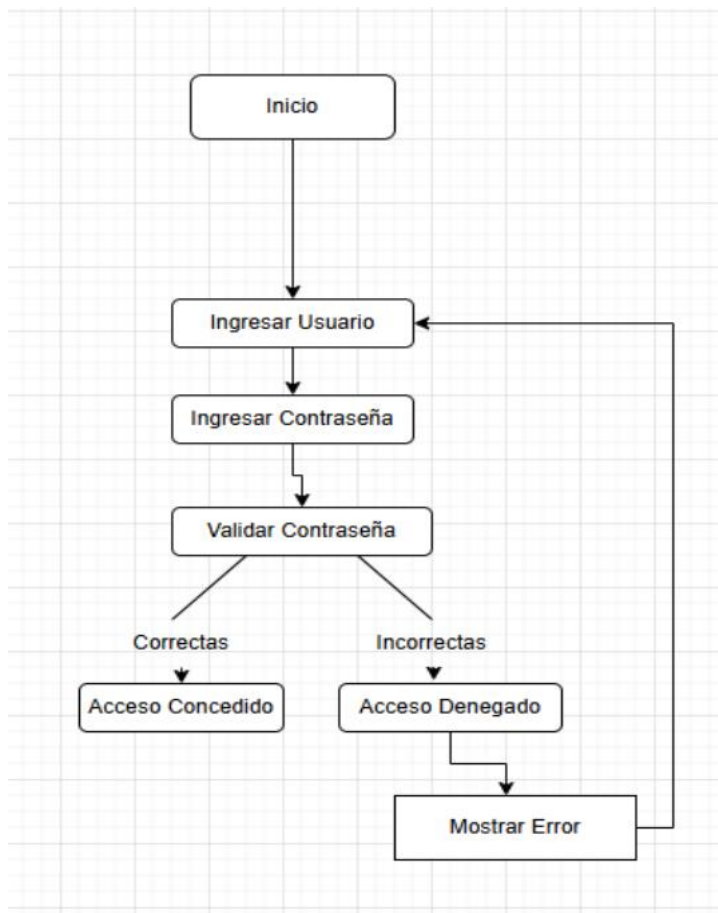
- Fecha de el repostaje
- Litros repostados
- Precio por litro
- Coste total
- Kilometraje en el momento de el llenado

El diagrama entidad–relación de *Kilómetro a Kilómetro* proporciona una representación clara del funcionamiento interno del sistema, mostrando cómo se almacenan los datos y cómo se relacionan las entidades entre sí, este modelo ofrece una base sólida para el cálculo de consumos, la visualización gráfica del gasto y la gestión eficiente de los vehículos de cada usuario.

Diagramas de Flujo

Login:

Diagrama de flujo para Registrar un usuario y acceder a la aplicacion:



Agregar Vehiculo:

Si el usuario registra un coche invalido o que no esta reconocido se mostrara como error y se volvera a pedir los datos para que los pueda ingresar correctamente, si estos datos son validos se le agrerara el vehiculo y podra llevar su control perfectamente.

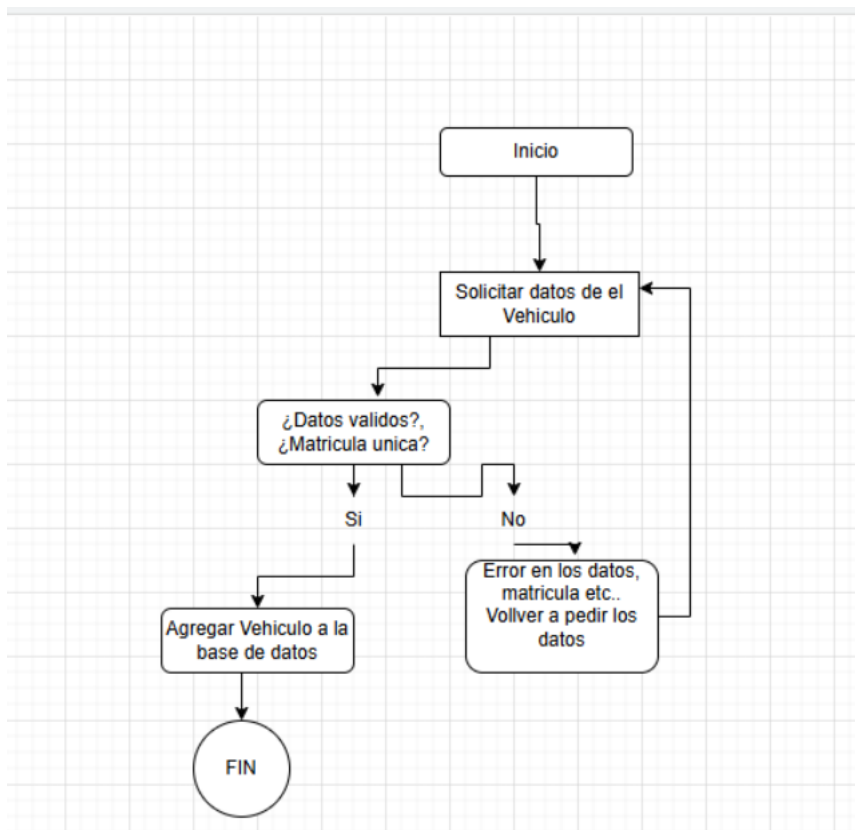


Diagrama de Flujo Registrar Repostaje

Se registraran los datos necesarios para el calculo de el repostaje de el vehiculo si estos son validos se le calculara el consumo, se registrara en la base de datos y actualizara las estadisticas de el vehiculo, si algun dato no es correcto se le volvera a pedir al usuario que introduzca los datos necesarios.

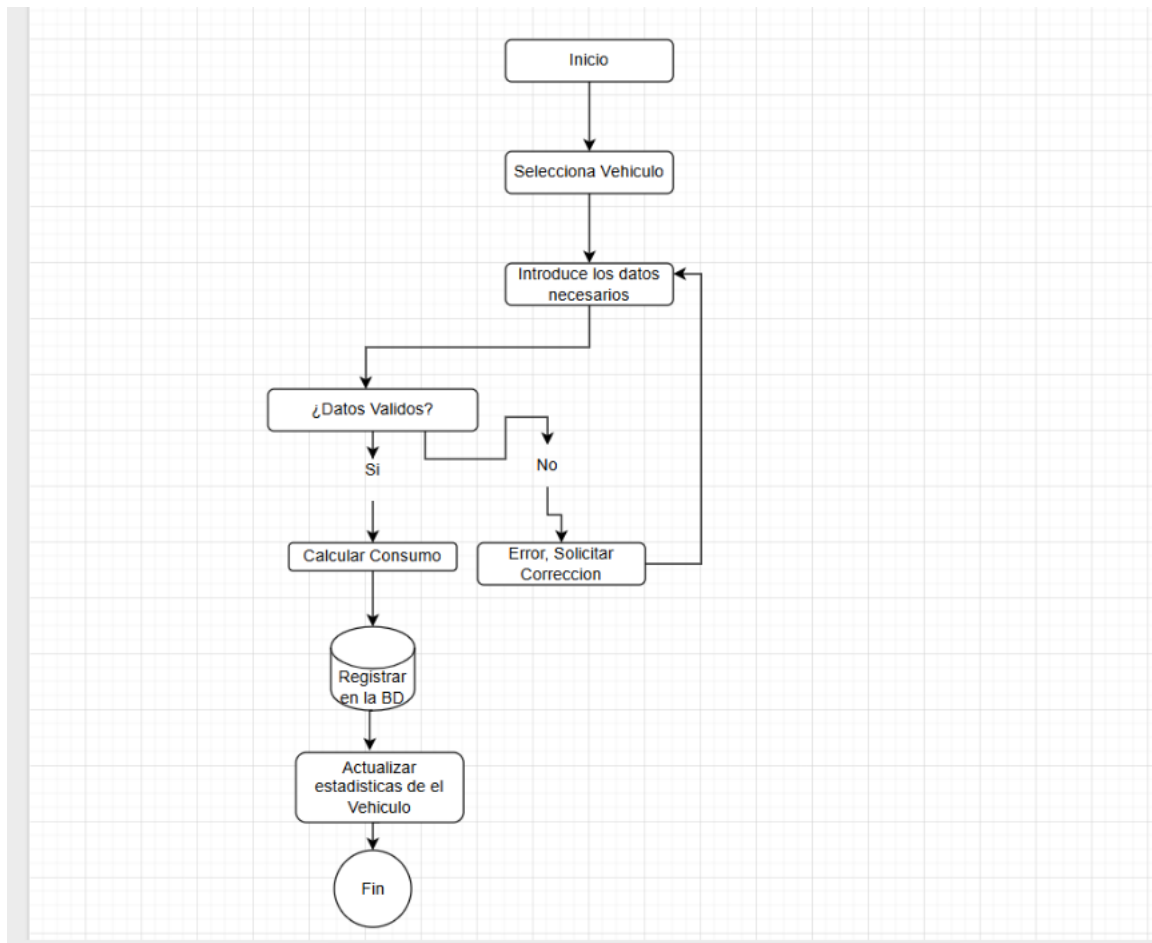


Diagrama de flujo de estadísticas:

El usuario accede al modulo de estadísticas, el sistema recupera los repostajes de el vehiculo, calculo de el consumo medio y genera graficos y estadísticas en la Intefaz.

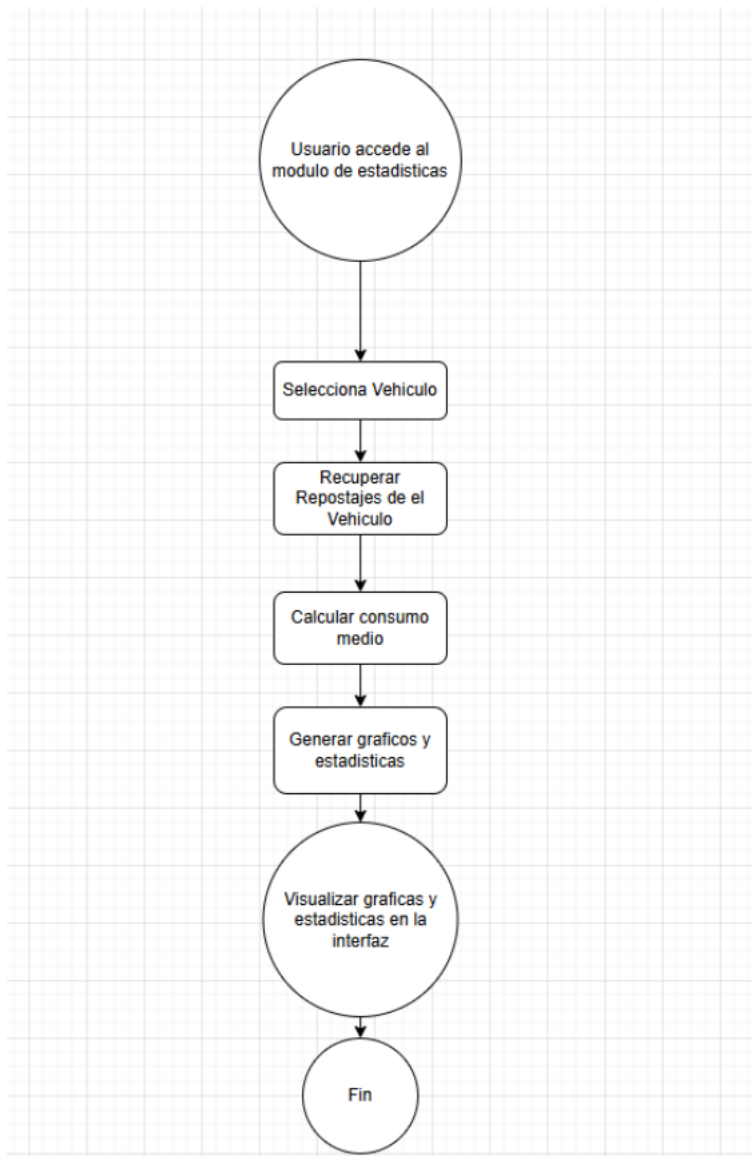
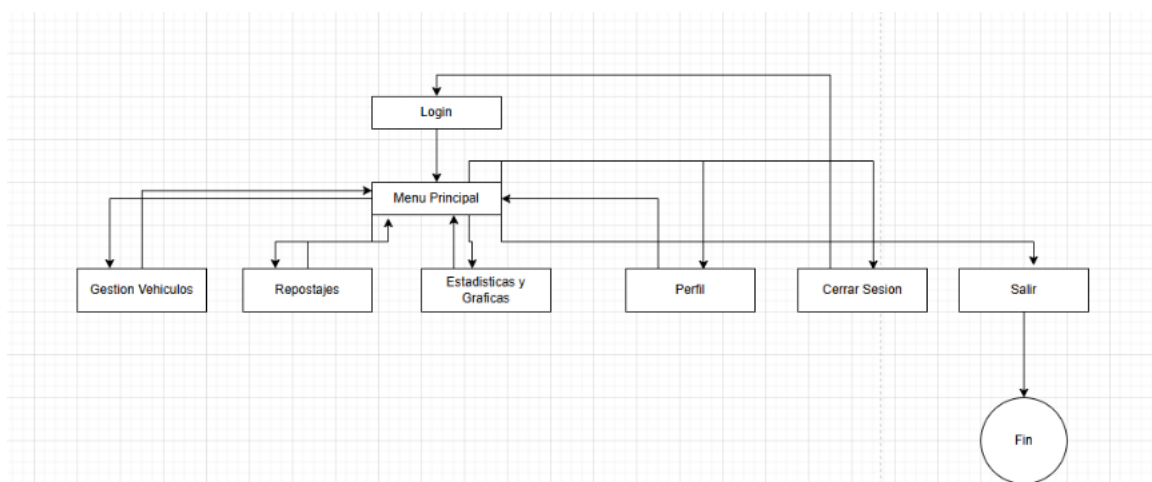


Diagrama de flujo de Navegacion.



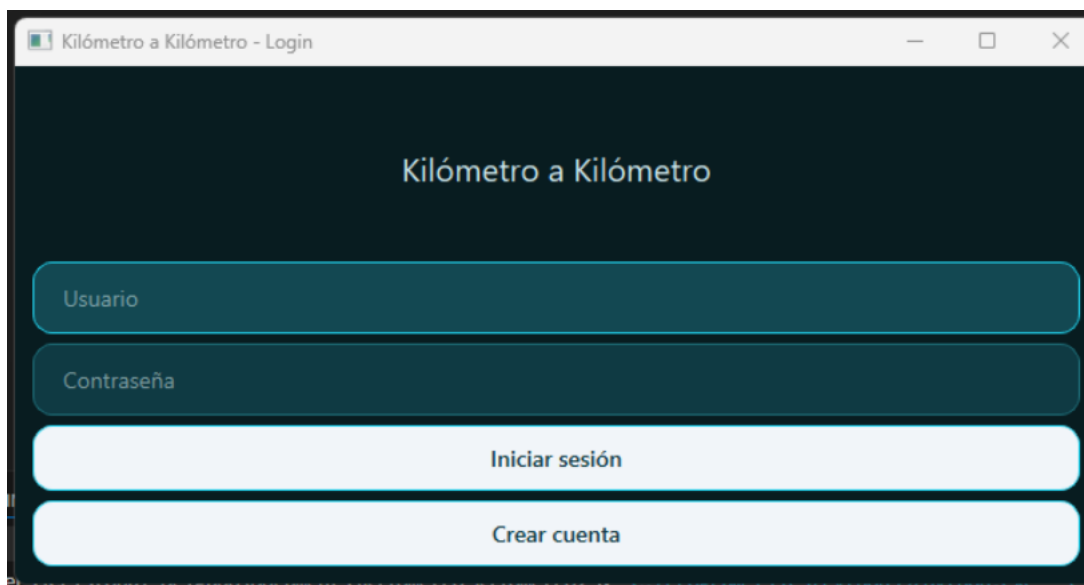
10.2 Interfaces:

Mockups para app de escritorio:

En este apartado se presentan los mockups de las principales interfaces de la aplicación Kilómetro a Kilómetro.

Estos diseños representan la estructura visual de la aplicación y sirven como referencia para el desarrollo de las pantallas definitivas implementadas con PySide6 y Qt Designer.

Pantalla de inicio de sesión:



- Permite al usuario introducir contraseña y nombre de usuario.
- Incluye botón de acceso y enlace a registro.
- Se prioriza una interfaz simple y clara para facilitar el acceso.

Registro de usuario:

Kilómetro a Kilómetro - Registro

Registrar Usuario

Nombre

Apellidos

Usuario

Email

Teléfono

01/01/2000

Selecciona ciudad

Contraseña

Repetir contraseña

Registrar

Ya tengo cuenta

- Permite al usuario introducir sus datos y registrarse

Pantalla Principal:



- Acceso rápido a vehículos, repostajes y estadísticas.
- Diseño limpio para facilitar la navegación.

Pantalla de gestion de vehiculos:

Mis vehículos							
Selecciona el vehículo activo							
	Activo	Tipo	Marca	Modelo	Matrícula	Combustible	Consumo
1		Coche	Toyota	Corolla	4567JKL	Híbrido	4.2
2	✓	Coches	Audi	Q3	5678JRG	Gasolina	6.0
3		Coches	Alfa Romeo	Giulia	1234LNF	Gasolina	4.0

Volver
✓ Marcar como activo
Eliminar
Añadir vehículo

- Listado de vehículos asociados al usuario.
- Opciones para añadir y eliminar vehículos.
- Cada vehículo muestra información básica (marca, modelo, combustible).

Pantalla de agregar Vehículo:



Añadir vehículo

Selecciona tipo

Selecciona marca

Selecciona modelo

Matrícula

2020

Gasolina

0,0 L/100km

Guardar Cancelar

- Esta pantalla permite al usuario agregar su vehículo, pudiendo elegir su tipo, marca, matrícula , consumo y año.
- Una vez añadido el vehículo debe de registrarse en el panel de gestion de vehículos

Mis Repostajes

Todos los meses Todos los años Buscar

	Fecha	Litros	Precio	Kilómetros
1	2026-01-10	15.0	1.8	120
2	2026-01-10	65.0	1.9	234

Volver Exportar CSV Exportar PDF Eliminar Nuevo repostaje

- Aquí vemos la gestión de los repostajes, con una tabla que muestra los repostajes realizados
- El usuario podrá filtrar por mes y año para llevar un control más regulado y organizado
- Esta información podrá ser exportada a PDF y CSV para poder llevar un buen control
- También se podrá eliminar un repostaje por ejemplo si se ha introducido por error

Pantalla de añadir repostaje:

Añadir repostaje

01/01/2000

Litros

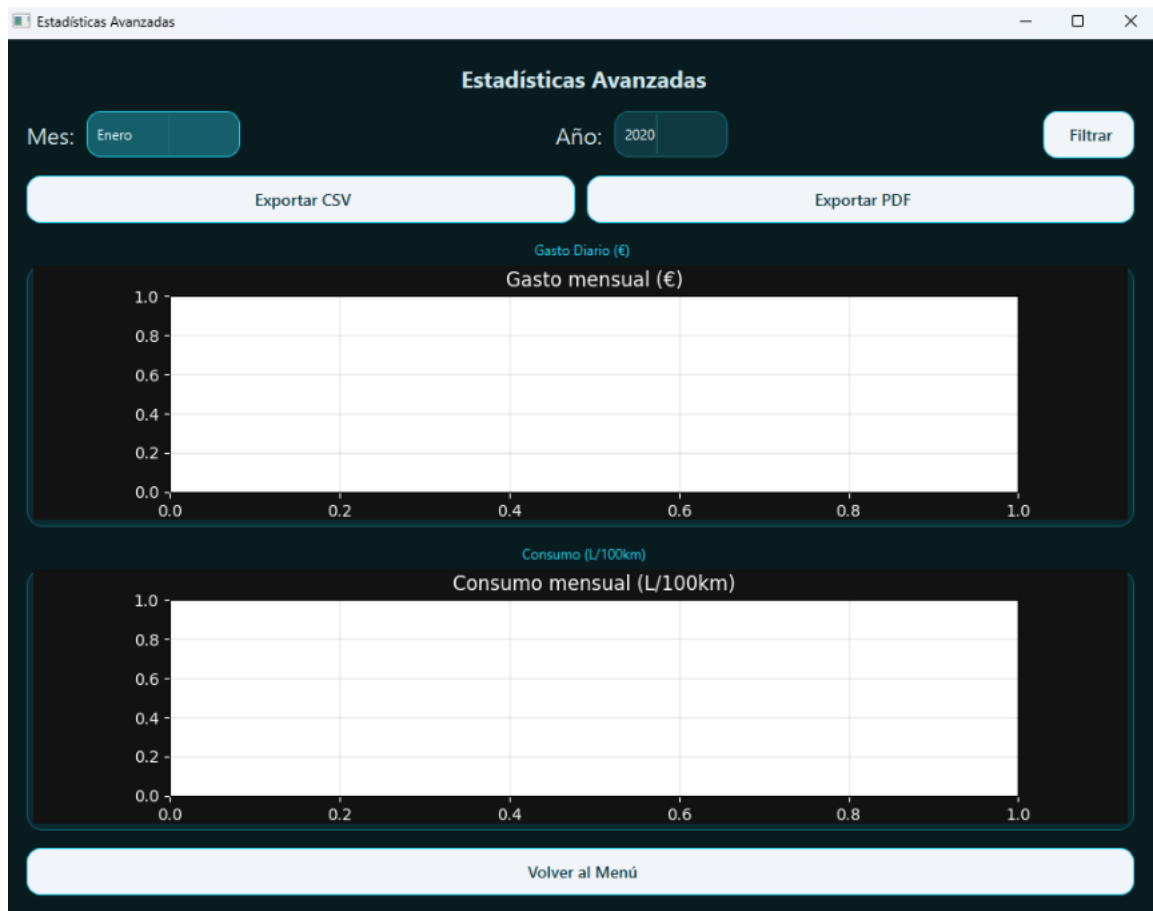
Precio total (€)

Kilómetros

Guardar Cancelar

- Aquí el usuario podrá registrar su repostaje en función a los kilómetros recorridos, litros , fecha de repostaje y precio al que esta el combustible necesario para su vehículo previamente registrado en la sección de vehículos
- Una vez registrado el repostaje, este aparecerá en la pantalla anterior registrado correctamente

Pantalla Estadísticas:



- Aquí aparecen 2 graficas una para el gasto mensual y la otra el consumo real calculado entre repostajes consecutivos
- El usuario podra filtrar por mes y año y posteriormente se le dibujara su grafica asociada al periodo seleccionado
- Tambien podra exportar el usuario a CSV y a PDF para poder tener su informacion guardada.

Pantalla de perfil de usuario:

Perfil de usuario

Perfil de usuario

Alvaro

Fernan

Garri

Fernan@gmail.com

678543123

Albacete

01/01/2000

Editar Guardar cambios

Cambiar contraseña

Nueva contraseña

Repetir nueva contraseña

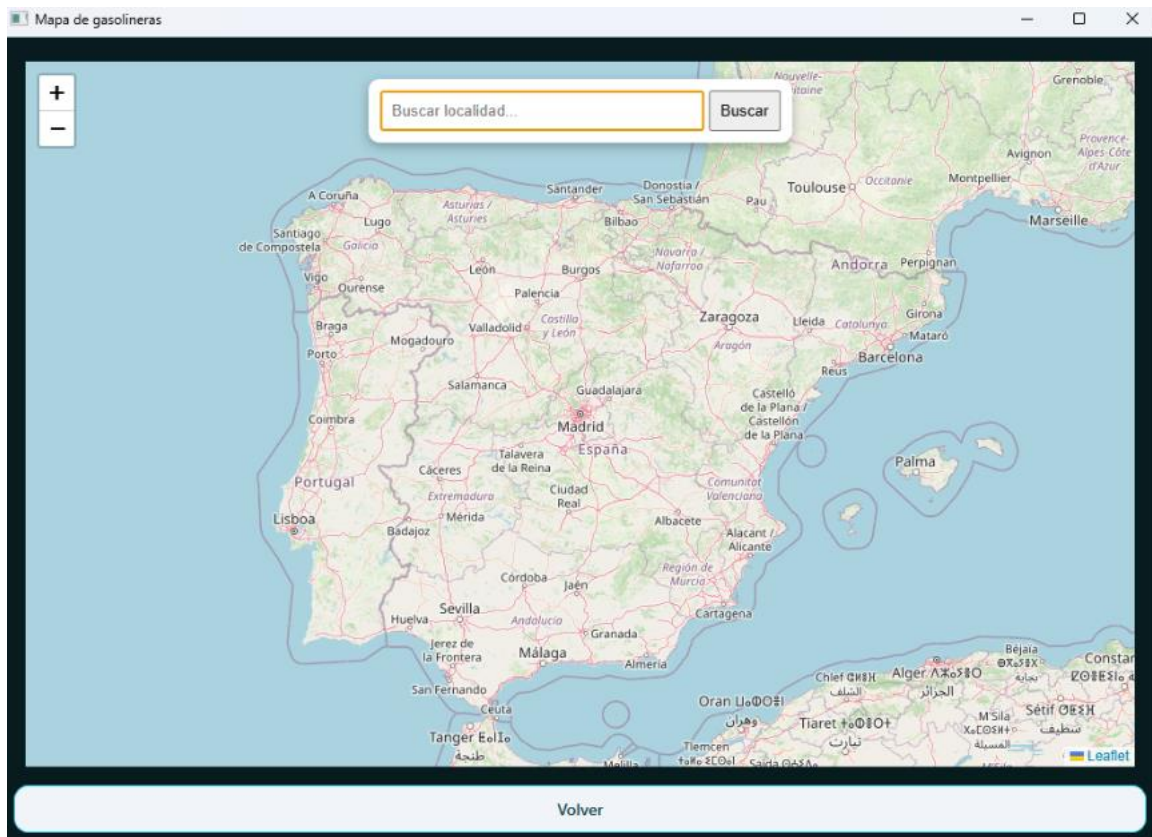
Guardar nueva contraseña

Eliminar cuenta

Volver

- Aquí el usuario podrá editar sus datos excepto el nombre de usuario
- Otra funcionalidad es el cambio de contraseña por si el usuario lo desea

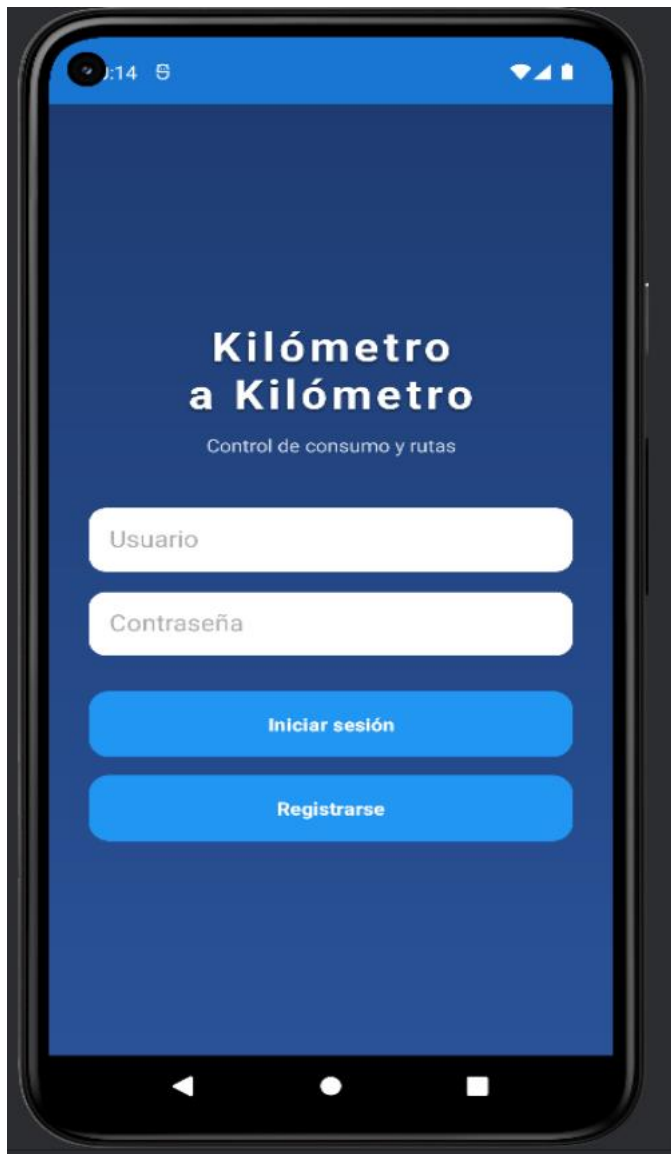
Pantalla de mapa de gasolineras:



- Esta pantalla carga un mapa, el cual muestra una barra para que el usuario pueda buscar la localidad que desee
- Este mapa muestra todas las gasolineras de la localidad seleccionada, con el precio de sus combustibles

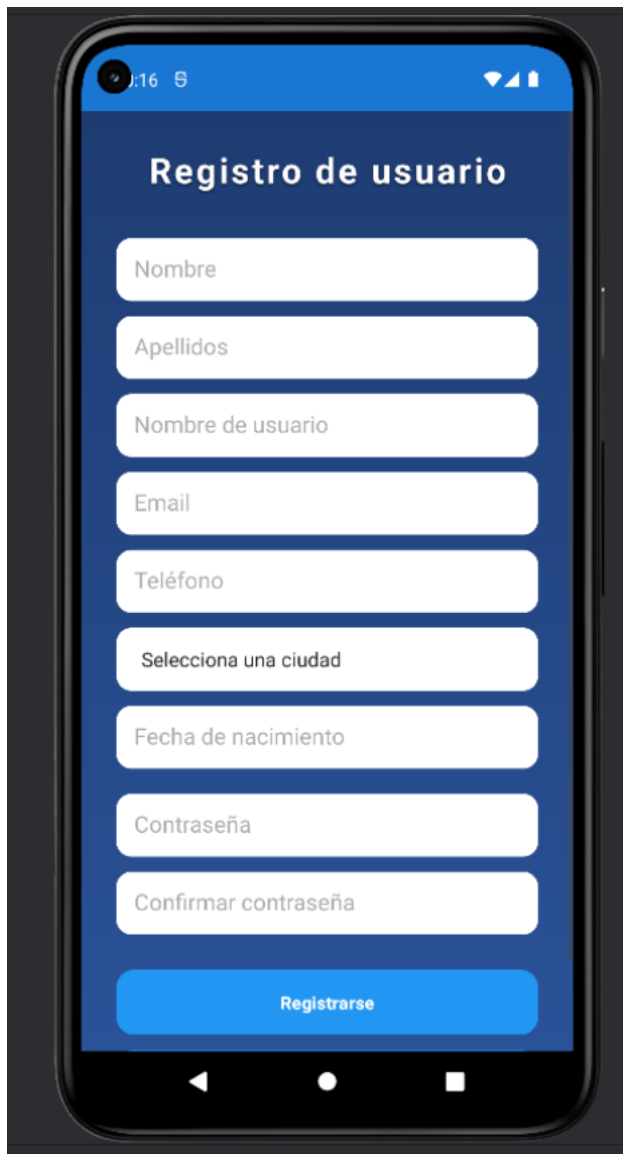
Mockups para la interfaz de Android (App movil)

Pantalla de inicio de sesion (Login)



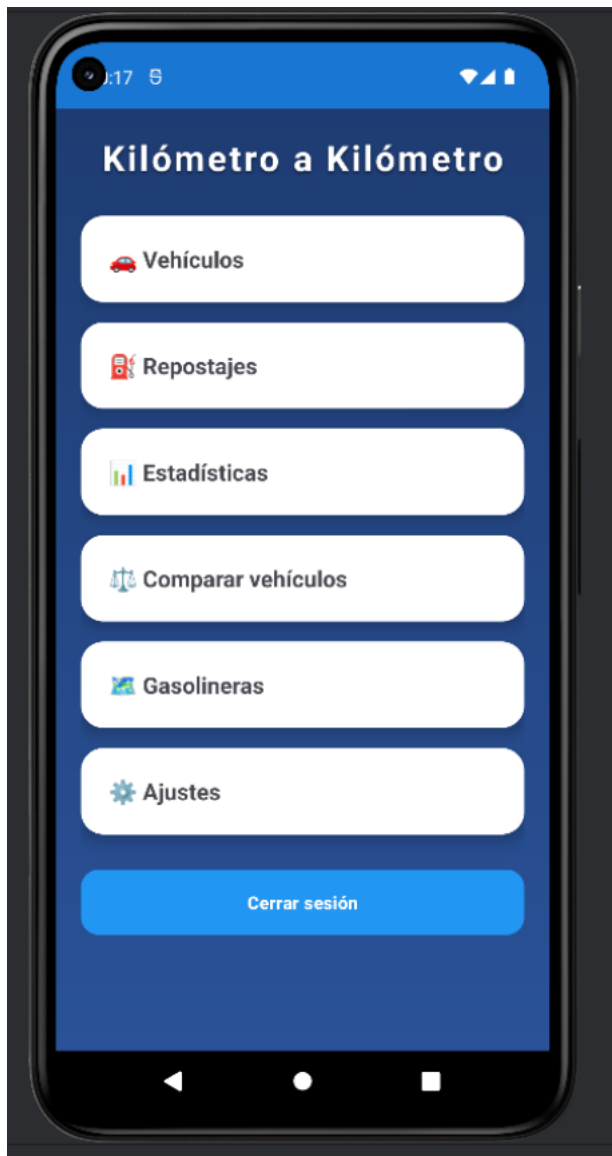
- El usuario podrá iniciar sesión con su nombre de usuario y una contraseña previamente registrada en la pantalla de registro de usuario.

Pantalla de registro de usuario



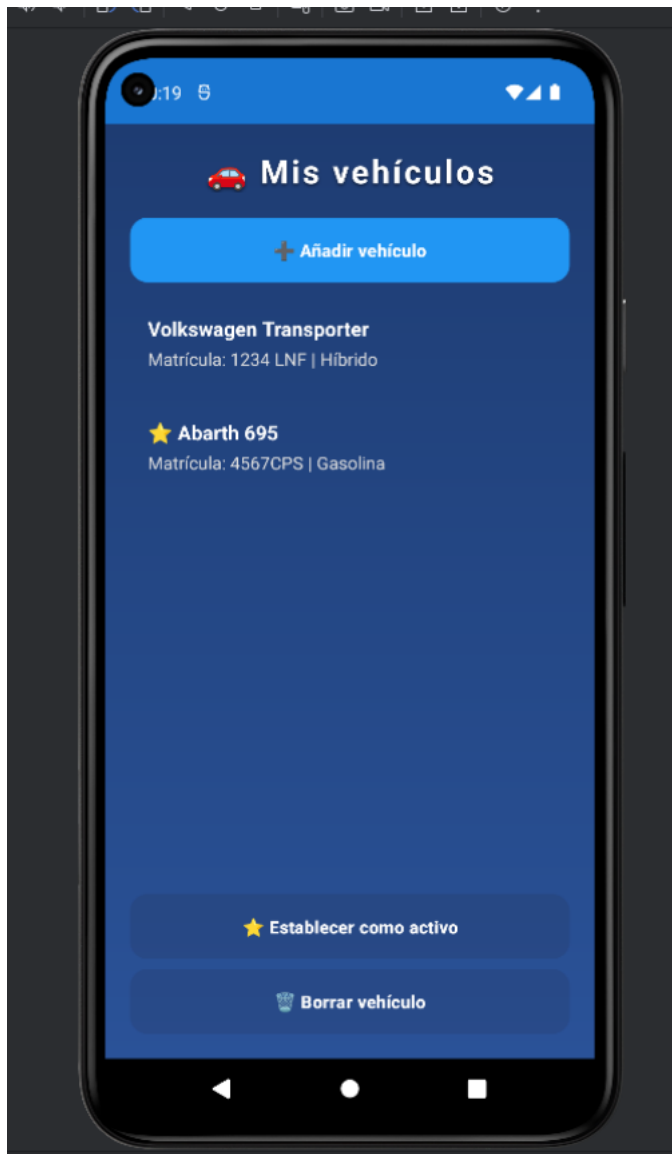
- Aquí tenemos la pantalla de registro de usuario, donde el usuario deberá de registrarse rellendo la informacion necesaria

Menu Principal



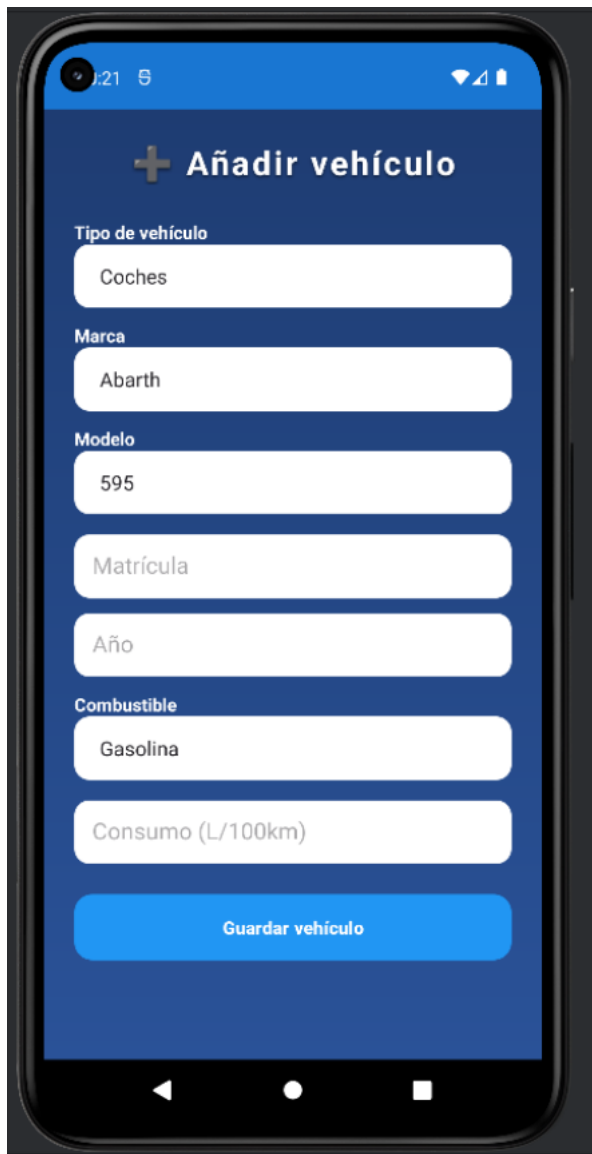
- Para el menu diseñamos una interfaz básica para que nuestro usuario tenga acceso a toda la informacion cuando lo desee.
- Cuenta con un boton de cerrar sesion por si nuestros usuarios registran una o mas cuentas

Gestion de vehiculos



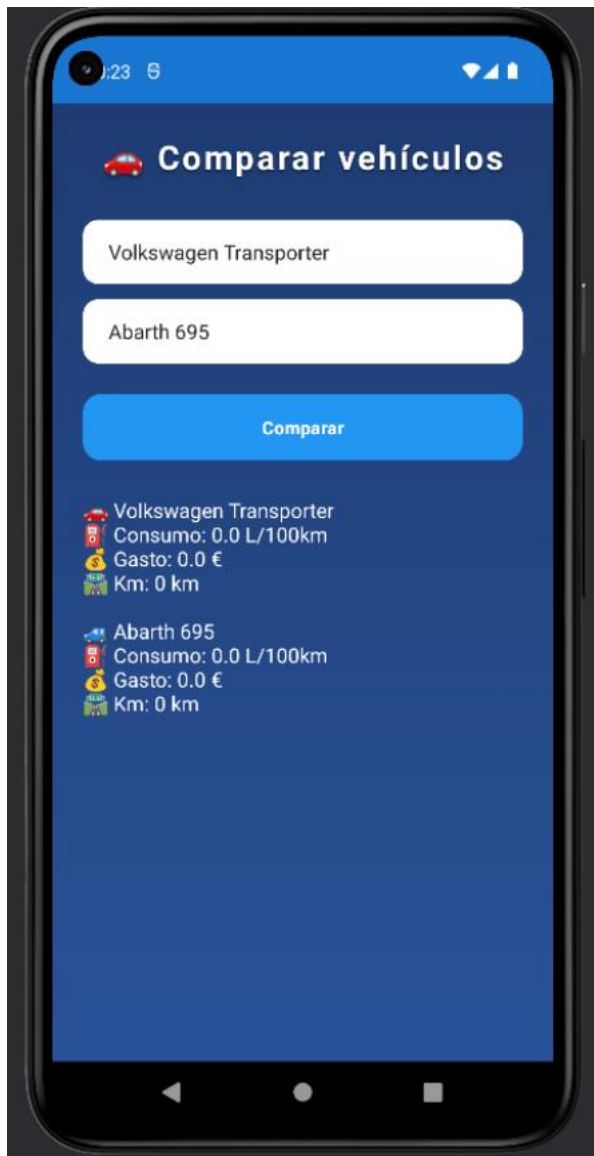
- El usuario debera de ver sus vehiculos registrados , contando con la opcion de marcar un vehiculo como activo , ya que puede resgistrar uno o varios vehiculos
- Tambien nuestros usuarios podran borrar el vehiculo que quieran

Añadir Vehiculos



- En esta pantalla es donde se registraran los vehiculos que tienen cada uno de los usuarios

Comparar Vehiculos



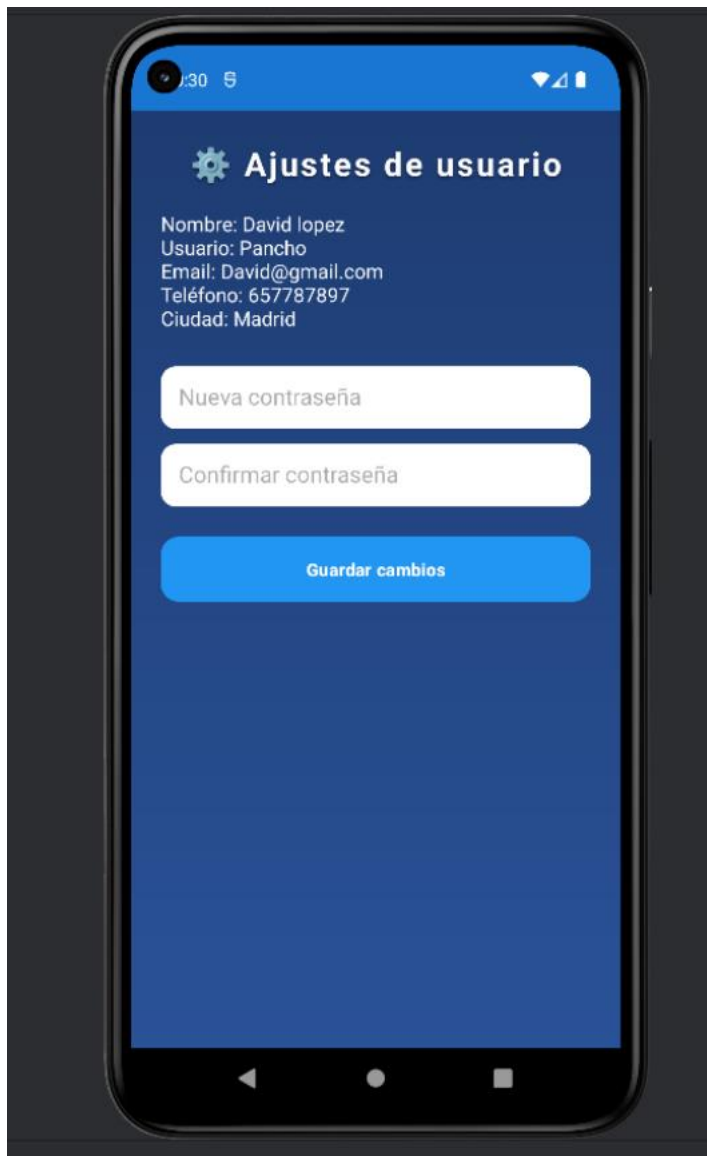
- En esta sección , los usuarios podran hacer una comparación de cada uno de sus vehiculos para ver cual le sale mas económico en ciertas ocasiones

Estadísticas



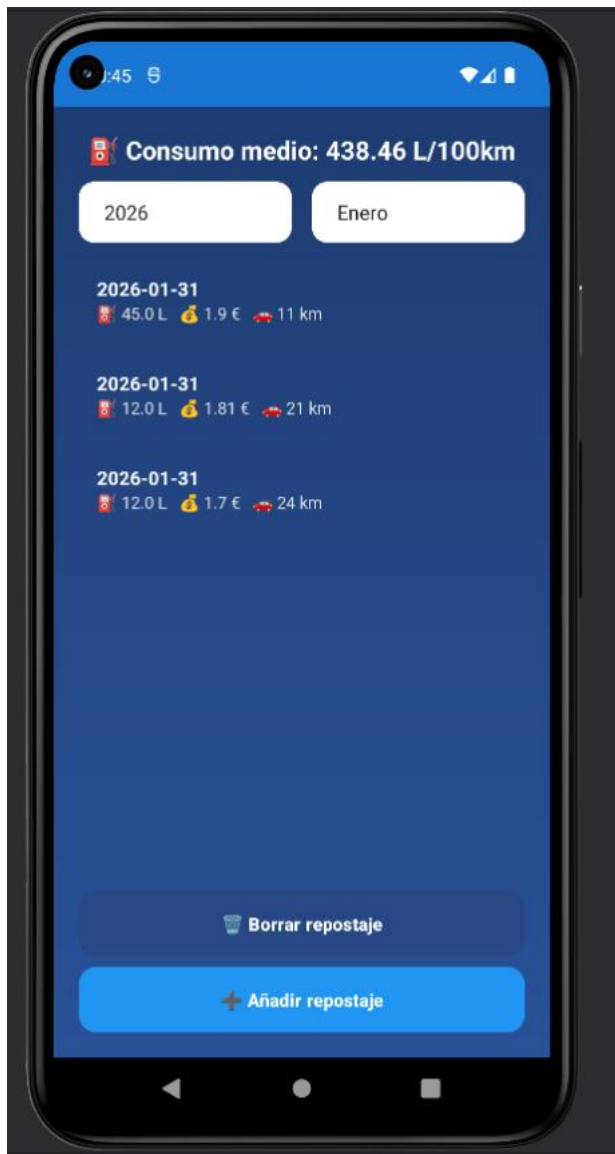
- Aquí se tendrá la opción de filtrado por mes y año y en función de las fechas elegidas se dibujarán tanto gráficas de consumo, como gráficas de gasto

Ajustes



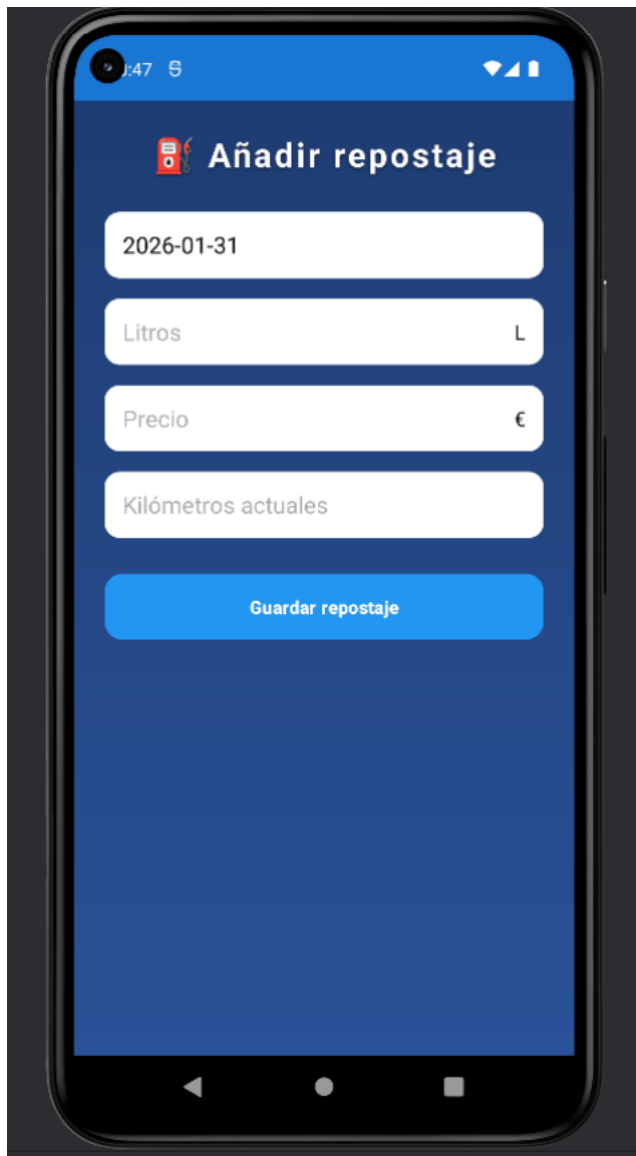
- Ventana para que nuestros usuarios puedan comprobar que sus datos están rellenos perfectamente
- Opcion por si se desea cambiar la contraseña

Repostajes



- Aquí vemos la pantalla de repostajes con 2 botones, uno para poder borrar el repostaje seleccionado y otro que nos lleva a la pantalla de añadir repostajes
- El usuario podrá llevar su registro por mes y año y arriba se le mostrara el consumo medio de ese mes y año

Añadir Repostajes



- En esta ventana se podrá añadir repostajes en función a la información necesaria como es la fecha, los litros, el precio de el combustible y los km actuales



- Aquí al igual que la función de la app de escritorio se podrá buscar por localidad de manera que en función a la localidad seleccionada esto cargará las gasolineras y sus precios actualizados

11. Metodología y estimación de costes:

Diagrama de Gantt:

El diagrama de Gantt muestra la planificación del proyecto *Kilómetro a Kilómetro* a lo largo de los meses de desarrollo. En él se puede ver en qué fechas se realiza cada fase del proyecto y cómo se distribuye el trabajo en el

tiempo. Es una forma visual de organizar el proyecto para asegurarnos de que todas las tareas se completan dentro del plazo establecido.

La planificación comienza con la fase de **Análisis**, donde se estudian las necesidades del proyecto y se definen los requisitos. Esta fase se realiza durante las tres primeras semanas de octubre.

Después continúa la fase de **Diseño**, en la que se crean los primeros bocetos de las interfaces y se diseña la base de datos. Esta parte se desarrolla entre finales de octubre y mediados de noviembre.

A partir de ahí empieza el **Desarrollo**, dividido en dos partes:

Backend y base de datos, donde se implementa SQLite, los modelos y la lógica interna del programa.

Interfaz (Frontend), donde se crean las pantallas con PySide6 y se conectan con el backend.

Estas tareas se llevan a cabo entre noviembre y diciembre.

Cuando todo el desarrollo está hecho, empieza la fase de **Pruebas y control de calidad**, donde se revisa que la aplicación funcione correctamente, se detectan errores y se corrigen. Esta fase ocupa desde finales de diciembre hasta mediados de enero.

Por último, el proyecto finaliza con la **Documentación y cierre**, que incluye redactar la memoria, elaborar el manual de usuario y preparar la presentación. Esta fase se desarrolla entre febrero y principios de marzo.

En resumen, el diagrama muestra de forma organizada el orden en el que se realizan las fases del proyecto, cuánto durará cada una y cómo se distribuye el trabajo a lo largo de los meses. Sirve como guía para controlar el progreso del desarrollo y asegurarse de que todo estará terminado dentro del plazo.

DIAGRAMA DE GANTT KILOMETRO A KILOMETRO

